

Estrutura de Dados II

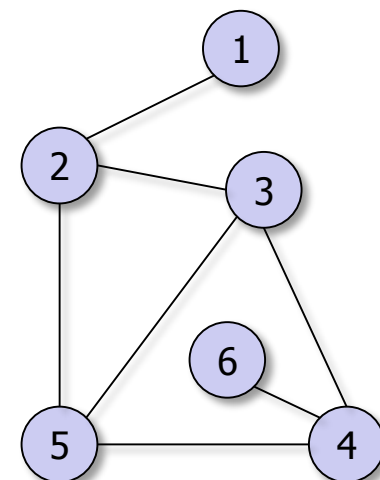
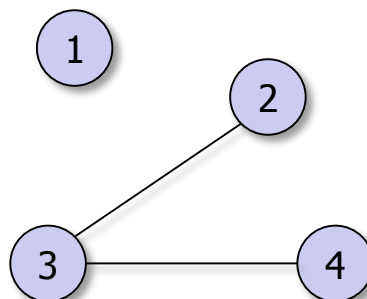
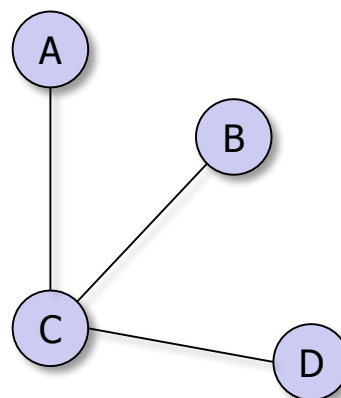
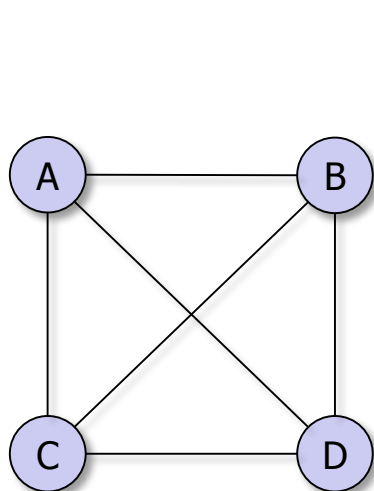
Grafos

Prof. Camilla Lobo Paulino

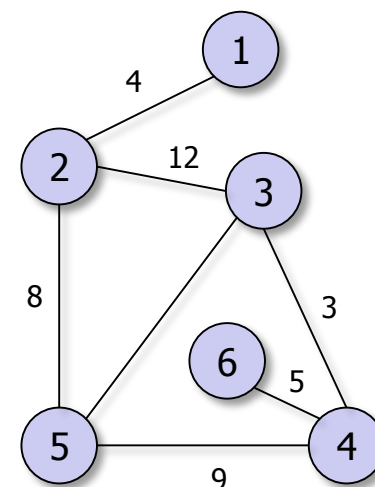
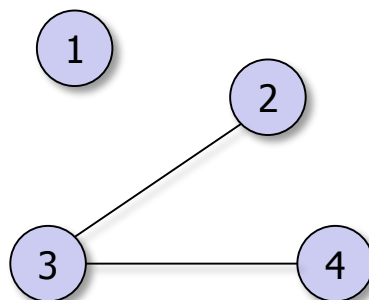
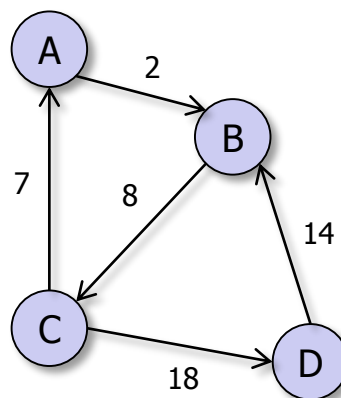
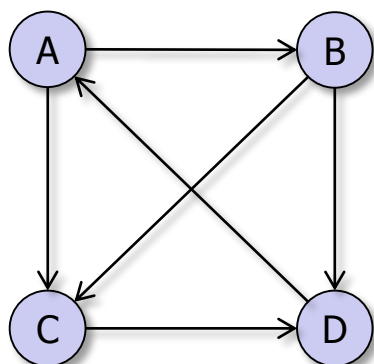
Camilla.paulino@uva.br

- O que é um grafo?
- Grafo é um elemento matemático formado por **vértices** (**nós**) e **arestas** (**ramos**).
- Dizemos que um grafo $G(V, A)$ é definido por:
 - ✓ V : um conjunto de vértices;
 - ✓ A : um conjunto de arestas que ligam $v, w \mid v, w \in V$.

- Visualmente, representamos os vértices como pontos ou círculos rotulados e as arestas como segmentos de retas ligando os vértices.

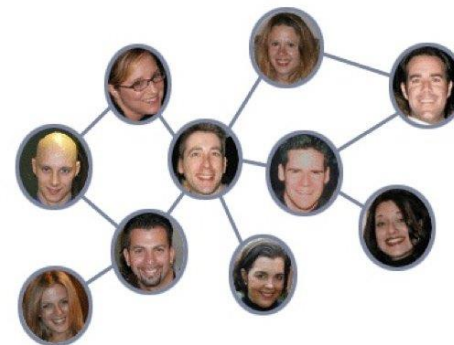
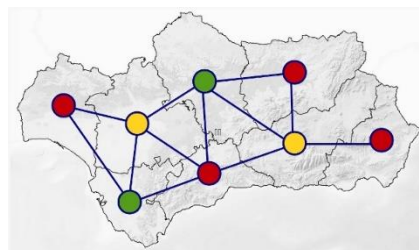
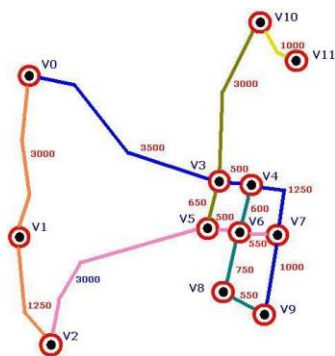
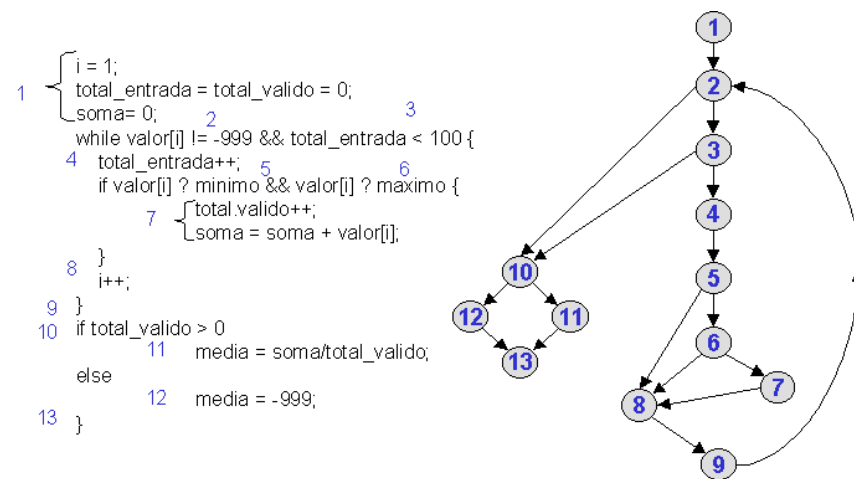


- Um grafo pode ser não orientado (não dirigido) ou orientado (dirigido).
- Um grafo pode ser ponderado ou não (pode ter pesos associados às arestas ou não).



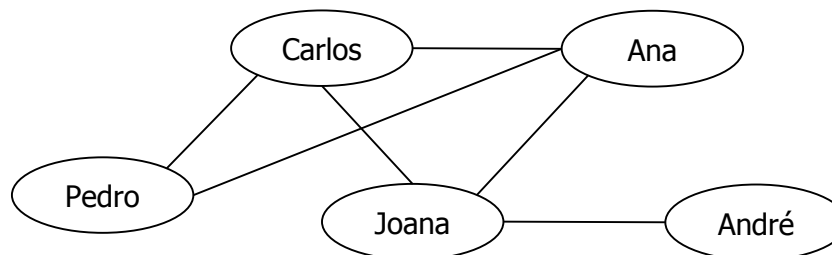
➤ Por que grafos são importantes? Várias situações do dia a dia podem ser representadas com grafos.

- ✓ Mapas de ruas, rodovias, etc.
- ✓ Fronteiras
- ✓ Circuitos e redes elétricas
- ✓ Sequenciamento de tarefas
- ✓ Redes de comunicação
- ✓ Redes de relacionamento
- ✓ Estruturas de programas



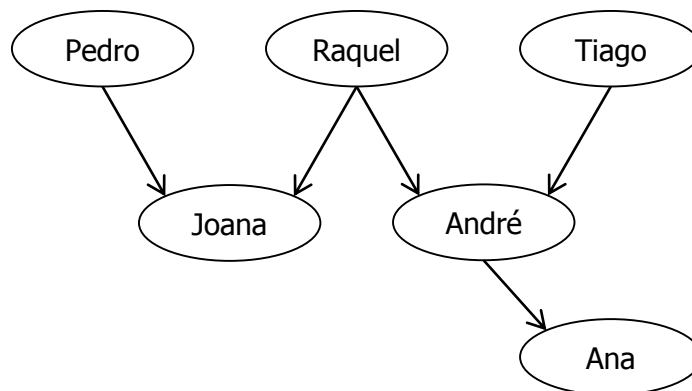
- Alguns problemas práticos que podemos resolver com grafos:
 - ✓ Transportar informações entre máquinas de uma rede.
 - ✓ Descobrir os melhores casamentos entre posições disponíveis em empresas e pessoas que se candidataram para essas posições
 - ✓ Descobrir grupos com interesses comuns em uma rede social.
 - ✓ Descobrir qual a rota mais curta para visitar cidades de uma região turística.
 - ✓ Criar roteiros de teste baseado em código-fonte (teste caixa-branca).

- Exemplo 1: grafo $G(V, A)$ não orientado
 - ✓ V = conjunto de pessoas.
 - ✓ A = conjunto de arestas que ligam vértices v, w | v é amigo de w .



- Note que as relações entre os vértices são **simétricas**:
 - ✓ Se v é amigo de w , então w é amigo de v .

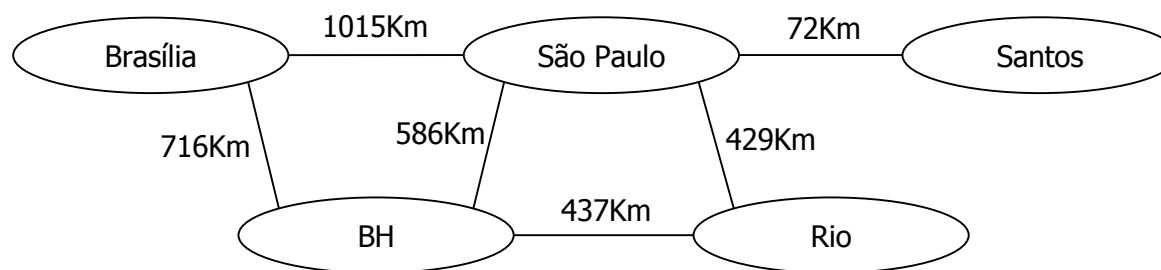
- Exemplo 2: grafo orientado $G(V, A)$
 - ✓ V = conjunto de pessoas.
 - ✓ A = conjunto de arestas que ligam vértices v, w | v é pai ou mãe de w .



- Note que as relações entre os vértices **não são simétricas**:
 - ✓ Se v é pai/mãe de w , então w não é pai/mãe de v .
- Grafos orientados também são chamados de **dígrafos**.

➤ Exemplo 3: grafo não orientado $G(V, A)$ ponderado

- ✓ V = conjunto de cidades.
- ✓ A = conjunto de arestas que ligam vértices v, w | é possível viajar de v até w .
- ✓ $p(v, w)$ = distância em quilômetros

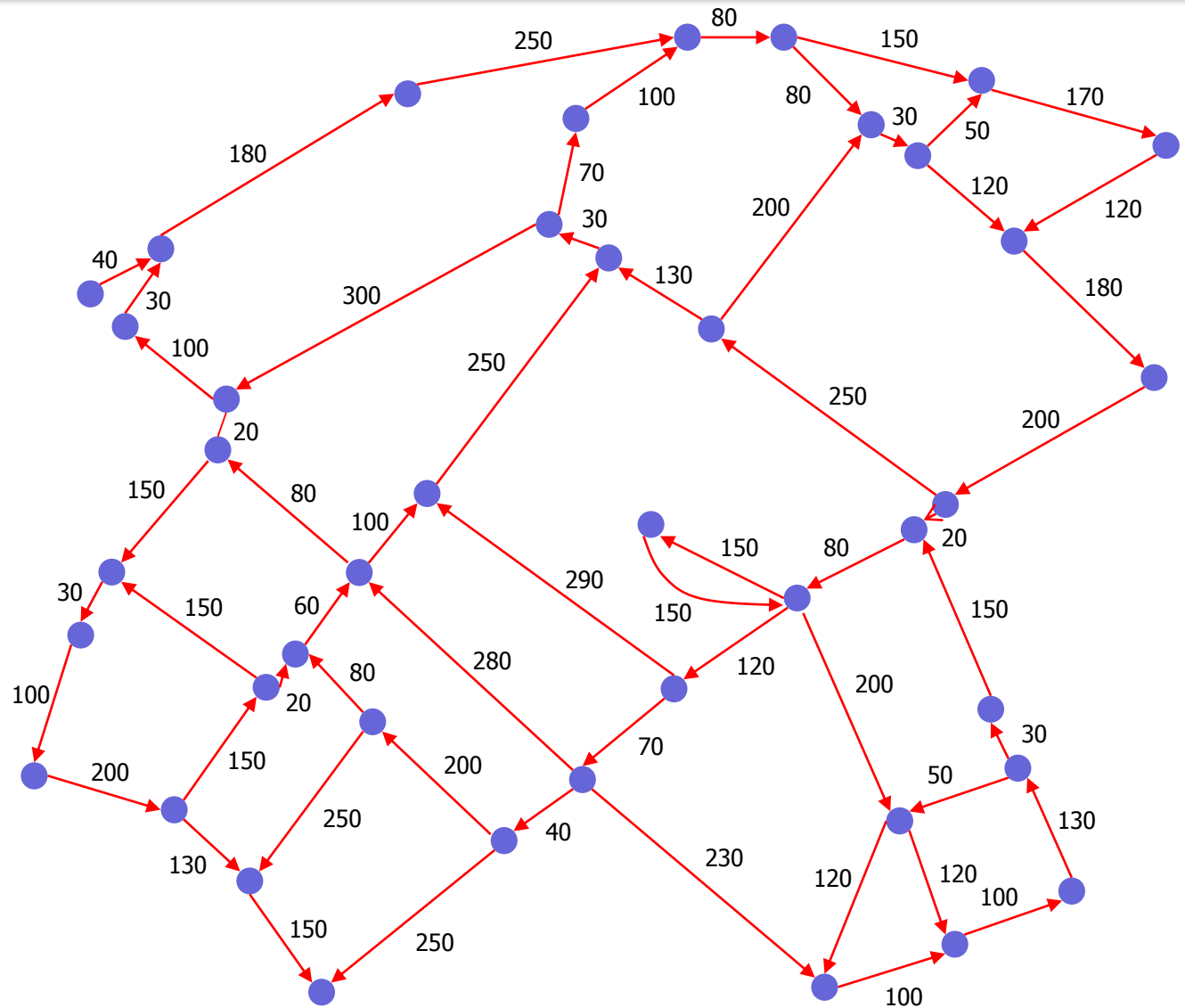


➤ Note que temos 2 **caminhos** de Brasília até o Rio:

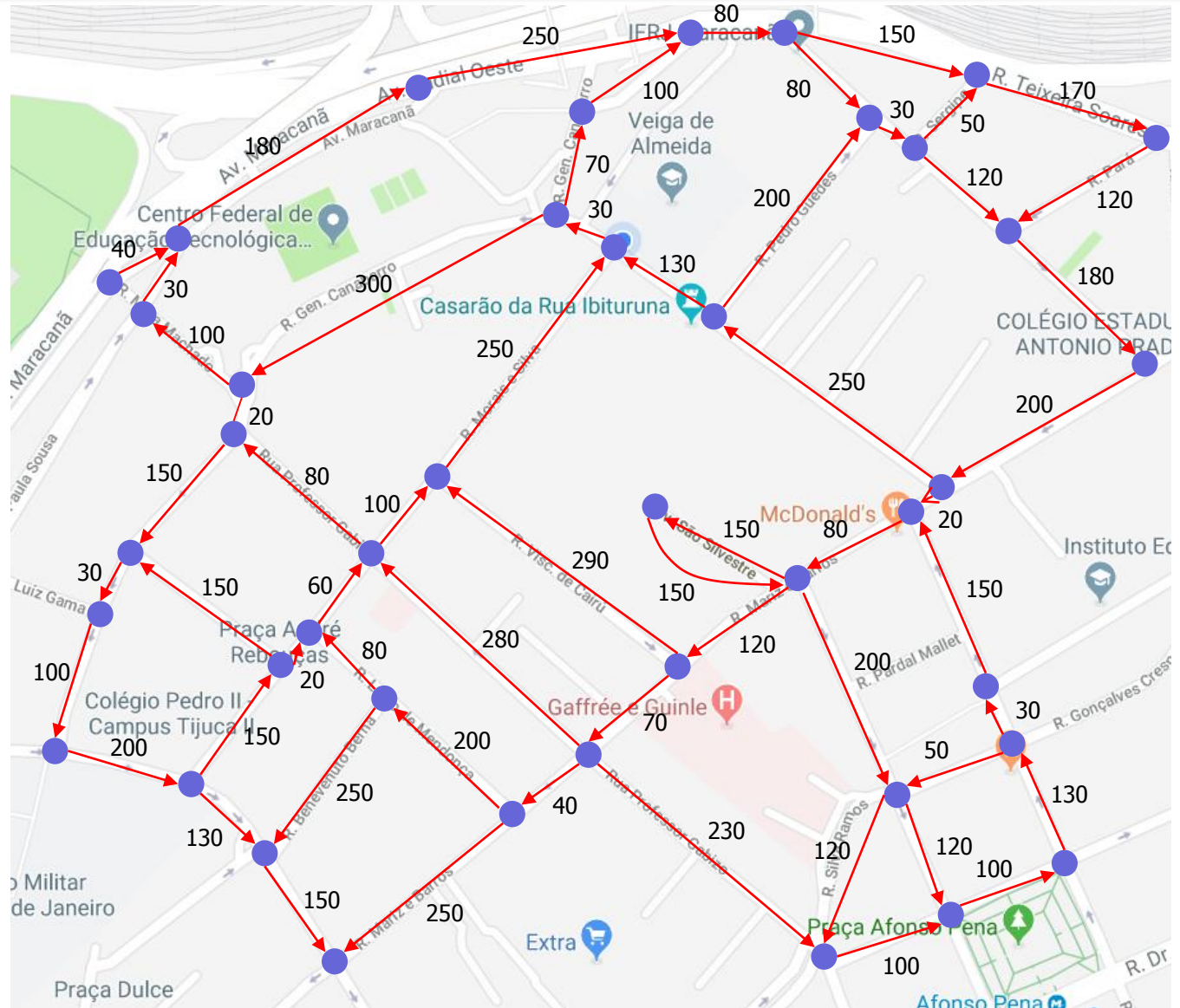
- ✓ Brasília-São Paulo-Rio: distância de 1444 Km
- ✓ Brasília-BH-Rio: distância de 1153 Km

- Exemplo 4: grafo orientado $G(V, A)$ ponderado
- ✓ V = conjunto de cruzamentos.
 - ✓ A = conjunto de arestas que ligam vértices v, w | é possível ir de v até w de carro.
 - ✓ $p(v, w)$ = distância em metros

➤ Exemplo 4:

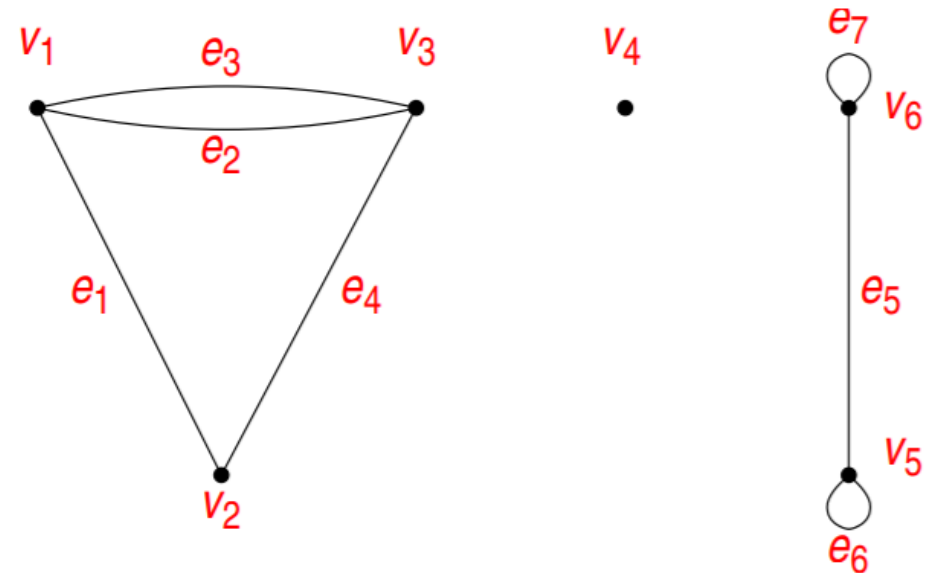


➤ Exemplo 4:

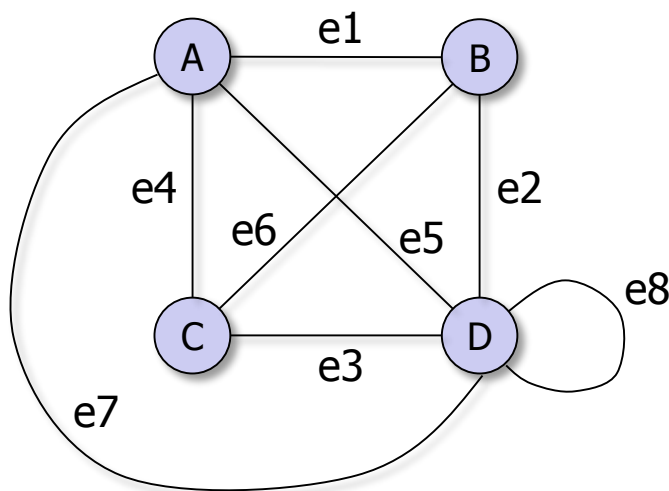


- Cada aresta está associada a um conjunto de um ou dois vértices, chamados nós terminais.
- Uma aresta conecta seus nós terminais.
- Uma aresta é dita ser incidente a cada um de seus nós terminais.
- Laço (loop): aresta com um único nó terminal.
- Arestas paralelas ou múltiplas: arestas associadas ao mesmo conjunto de vértices.
- Vértices adjacentes: vértices conectados por uma aresta.
- Um vértice que é nó terminal de um laço é dito ser adjacente a si próprio.
- Arestas adjacentes: arestas que incidem em um mesmo vértice.
- Um vértice que não possui nenhuma aresta incidente é chamado de isolado.
- Um grafo sem nenhum vértice é chamado de vazio.

- e_1 , e_2 e e_3 são incidentes a v_1 .
- v_2 e v_3 são adjacentes a v_1 .
- e_2 , e_3 e e_4 são adjacentes a e_1 .
- e_6 e e_7 são laços.
- e_2 e e_3 são paralelas.
- v_5 e v_6 são adjacentes entre si.
- v_4 é um vértice isolado.



- **Definição:** seja $G(V, A)$ um grafo e $v \in V$. O grau de v é igual ao número de arestas incidentes a v (caso a aresta seja um laço ela é contada duas vezes).
- O grau total de G é a soma dos graus de todos os vértices de G .



Grau(A): 4

Grau(B): 3

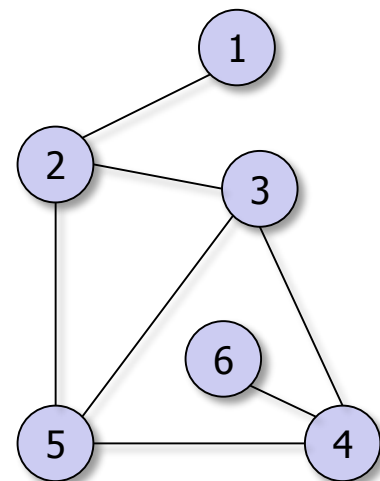
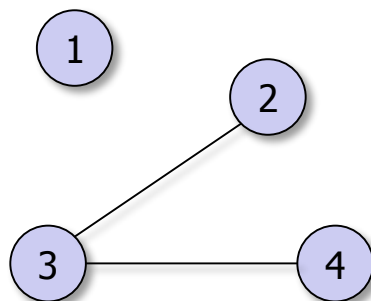
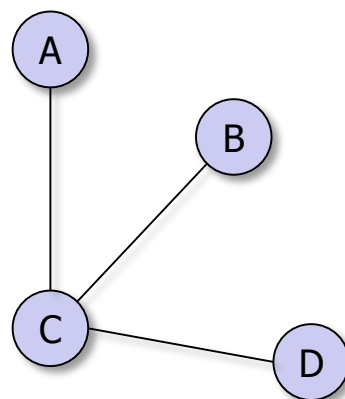
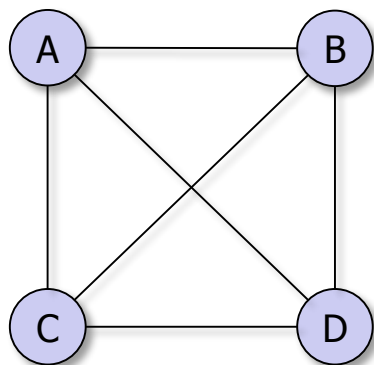
Grau(C): 3

Grau(D): 6

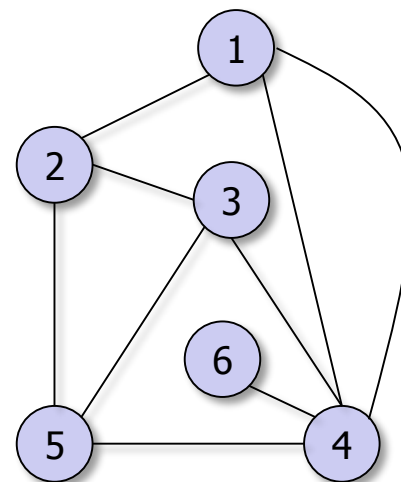
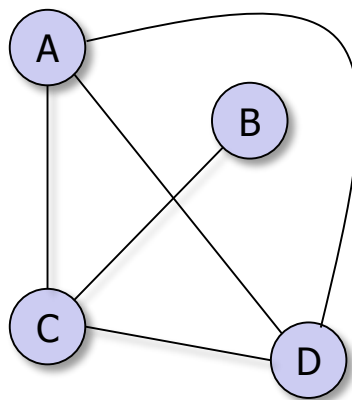
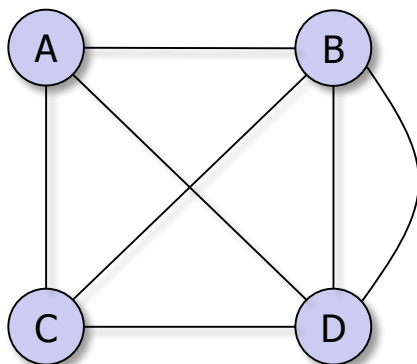
Grau do grafo: $4 + 3 + 3 + 6 = 16$

- **Observação:** caso o grafo seja orientado são contadas as arestas que chegam e que saem do vértice.

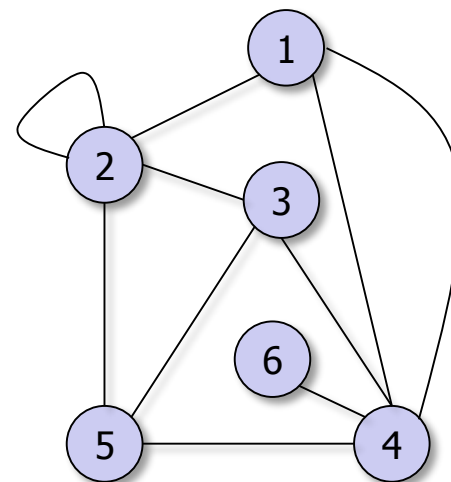
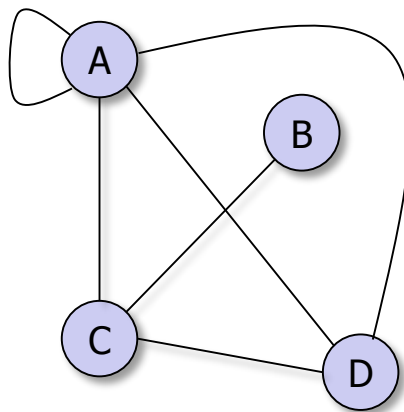
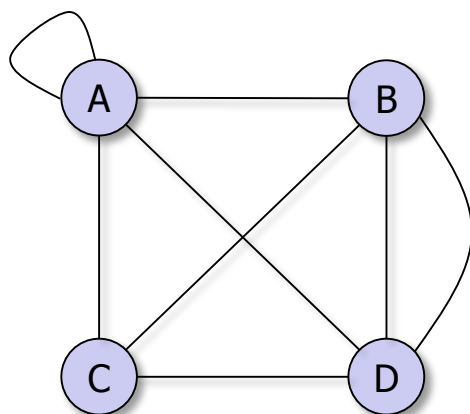
- **Grafo simples:** um grafo $G(V, A)$ é simples quando não possui **laço** nem **aresta paralela** (múltipla).



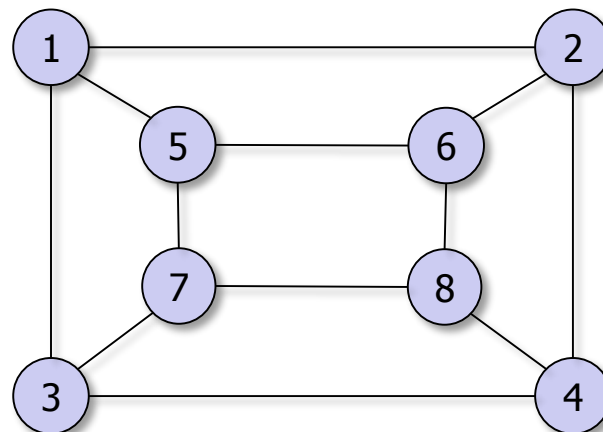
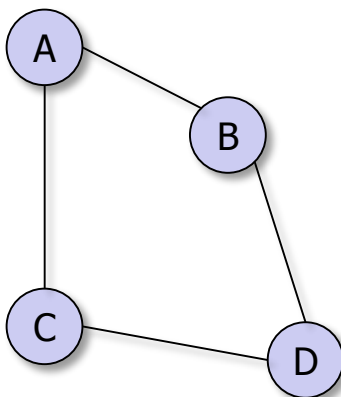
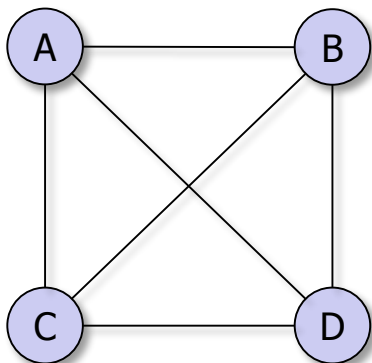
- **Multigrafo**: um grafo $G(V, A)$ é um multigrafo quando não possui **laço**, mas pode ter **arestas paralelas** (múltiplas).



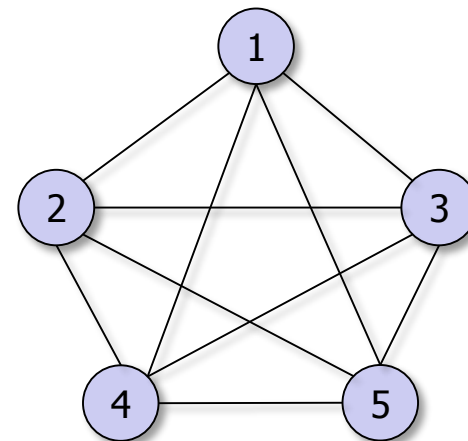
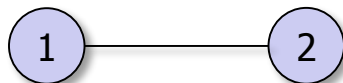
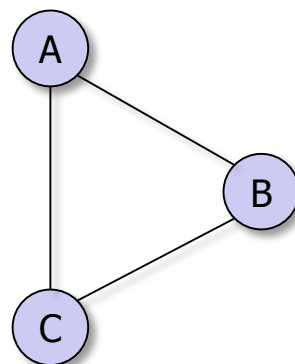
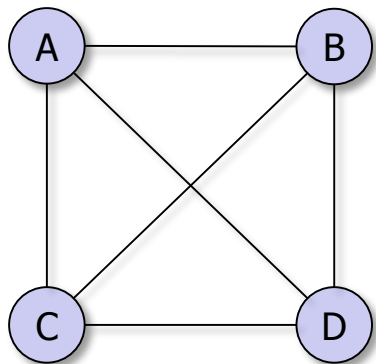
- **Pseudografo:** um grafo $G(V, A)$ é um pseudografo quando possui **laços** e **arestas paralelas** (múltiplas).



- **Grafo regular:** um grafo $G(V, A)$ é regular quando **todos** os seus vértices tem o **mesmo grau**.

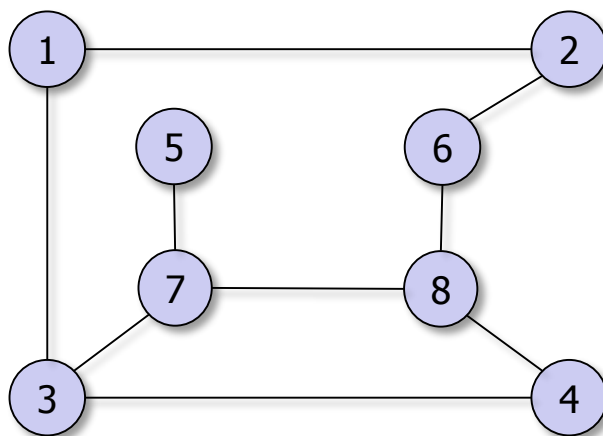


- **Grafo completo:** um grafo $G(V, A)$ é completo quando contém exatamente uma aresta para cada par distinto de vértices.



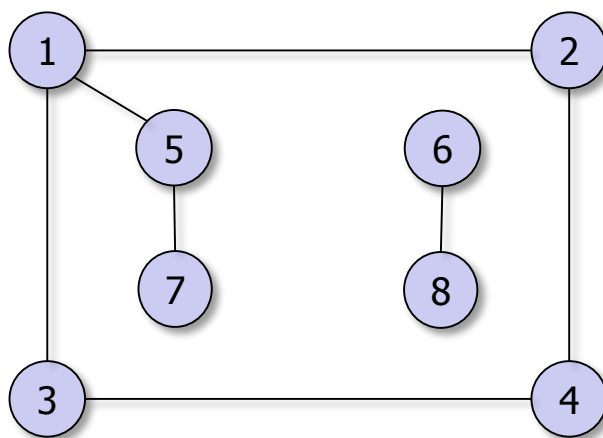
- Quantidade de vértices: $\frac{V^2 - V}{2}$

- **Grafo conexo**: um grafo $G(V, A)$ é **conexo** quando existe um conjunto de arestas adjacentes que liga um par qualquer de vértices, ou seja, dados dois vértices quaisquer v e $w \in V$, existe um conjunto de arestas adjacentes que ligam v a w .

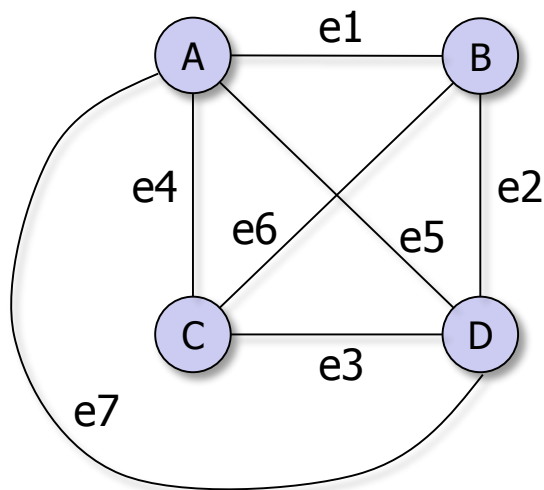


- Outra definição: um grafo $G(V, A)$ é **conexo** quando, dados dois vértices quaisquer v e $w \in V$, existe um **caminho** de v para w .

- **Grafo desconexo**: um grafo $G(V, A)$ é **desconexo** quando existe ao menos um par de vértices para os quais não existe um conjunto de arestas adjacentes que os ligam, ou seja, existe v e $w \in V$, tal que não existe um conjunto de arestas adjacentes que ligam v a w .

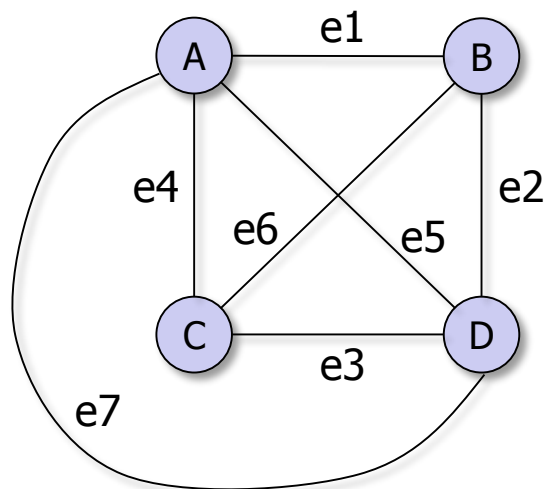


- Seja $G(V, A)$ um grafo não orientado e $v, w \in V$.
- **Passeio** (walk): um passeio de v para w é uma **sequência alternada de vértices e arestas adjacentes** de G , que inicia no vértice v e termina no vértice w .



Passeio de C a D:
C e6 B e2 D e7 A e1 B e2 D

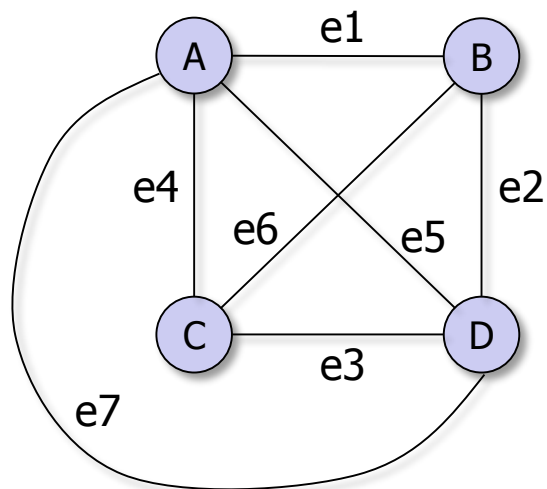
- **Passeio fechado** (closed walk): é um passeio que inicia e termina no mesmo vértice.



Passeio fechado em C:

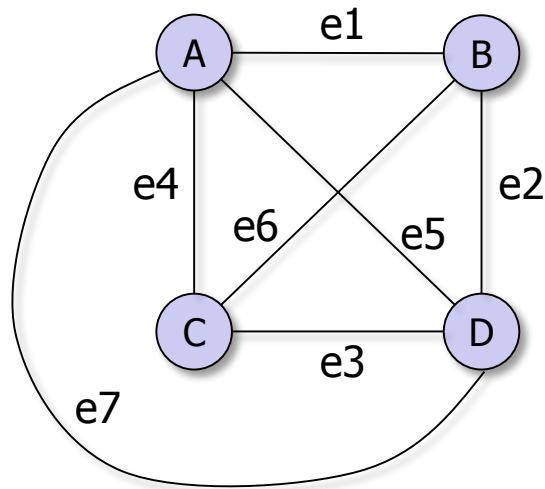
C e6 B e2 D e7 A e1 B e2 D e3 C

- Caminho (path): é um passeio onde todos os **vértices** são **distintos**.



Caminho de C a D:
C e6 B e2 D e7 A e5 D

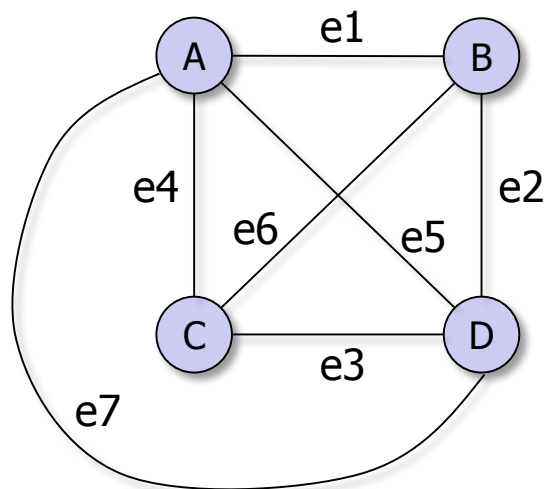
- **Caminho fechado** (closed path): é um caminho onde o vértice inicial e final é o mesmo.



Caminho fechado iniciando em C:
C e6 B e2 D e7 A e1 B e2 D e3 C

- Um caminho fechado com pelo menos uma aresta é chamado de **ciclo**.

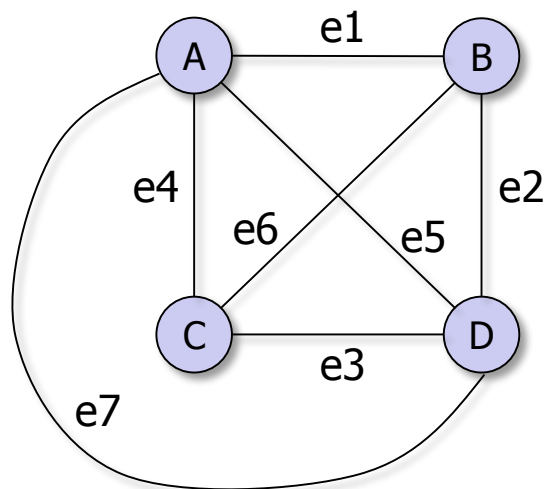
- **Trajeto ou Trilha (trail):** é um passeio de v para w onde todas as arestas são distintas.



Trajeto de C a D:

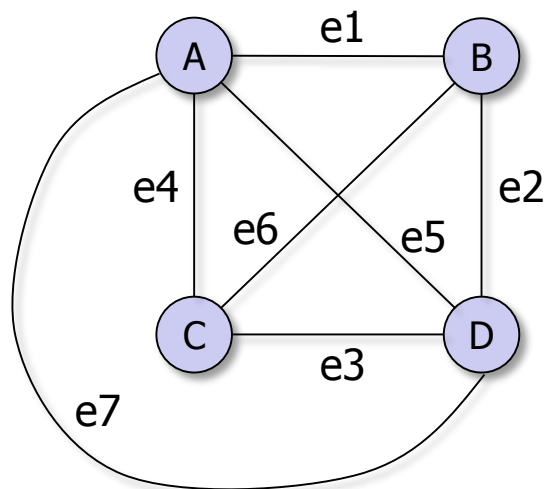
C e6 B e2 D e7 A e4 C e3 D

- **Trajeto Simples** (simple trail): um caminho de v para w sem repetição de arestas ou vértices.



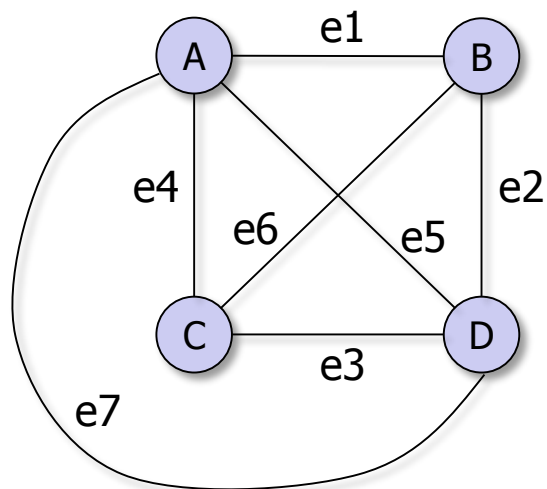
Trajeto simples de C a A:
C e6 B e2 D e5 A

- **Circuito** (circuit): é um trajeto fechado, ou seja, é um trajeto onde os vértices inicial e final são idênticos.



Circuito iniciando em C:
C e6 B e2 D e7 A e1 B e5 C

- **Circuito Simples** (simple circuit): trajeto simples fechado, ou seja, um caminho sem repetição de arestas ou vértices e onde os vértices inicial e final são idênticos.



Circuito simples iniciando em C:
C e6 B e2 D e7 A e4 C

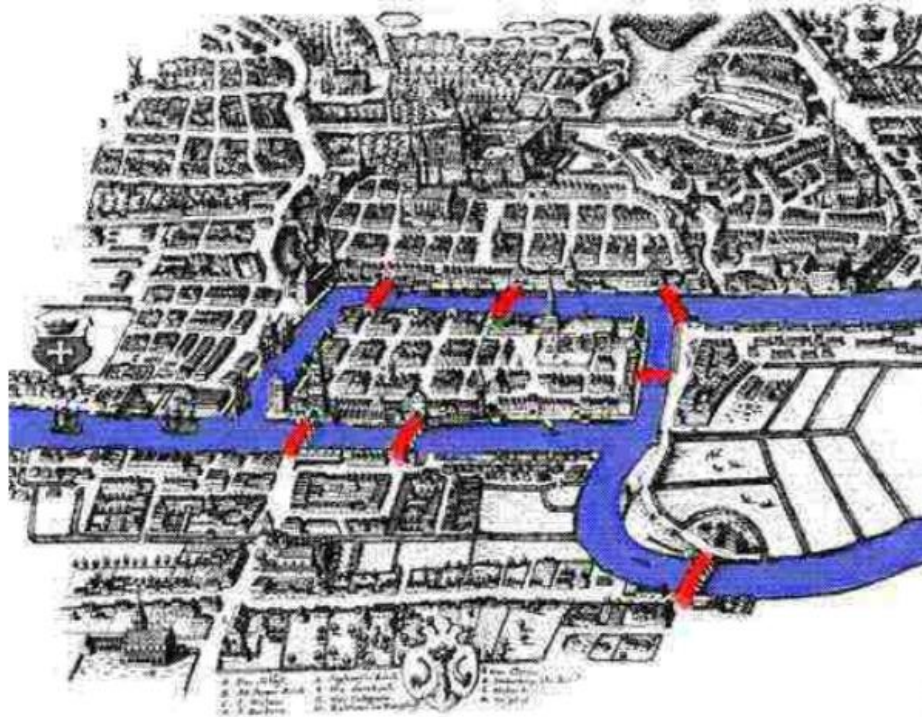
- Um circuito simples também é chamado de **ciclo simples**.

➤ Resumo dos Caminhamentos:

Tipo	Aresta repetida?	Vértice repetido?	Inicia e termina no mesmo vértice?
Passeio (walk)	Pode	Pode	Pode
Passeio fechado (closed walk)	Pode	Pode	Obrigatoriamente
Caminho (path)	Pode	Não	Pode
Caminho fechado (closed path) ou Ciclo	Pode	Somente inicial e final	Obrigatoriamente
Trajeto ou Trilha (trail)	Não	Pode	Pode
Trajeto fechado ou Circuito (circuit)	Não	Pode	Obrigatoriamente

- Importante: note que, na prática, o caminho também não tem aresta repetida, pois os vértice não pode ser repetido.

- A cidade de Königsberg foi construída numa região onde haviam dois braços do Rio Pregel e uma ilha. Foram construídas sete pontes ligando diferentes partes da cidade.

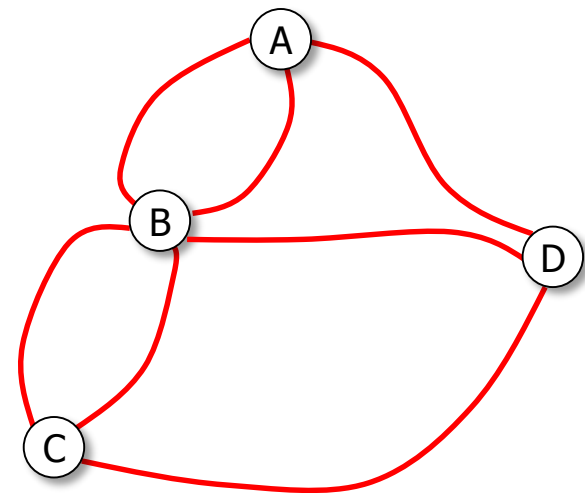
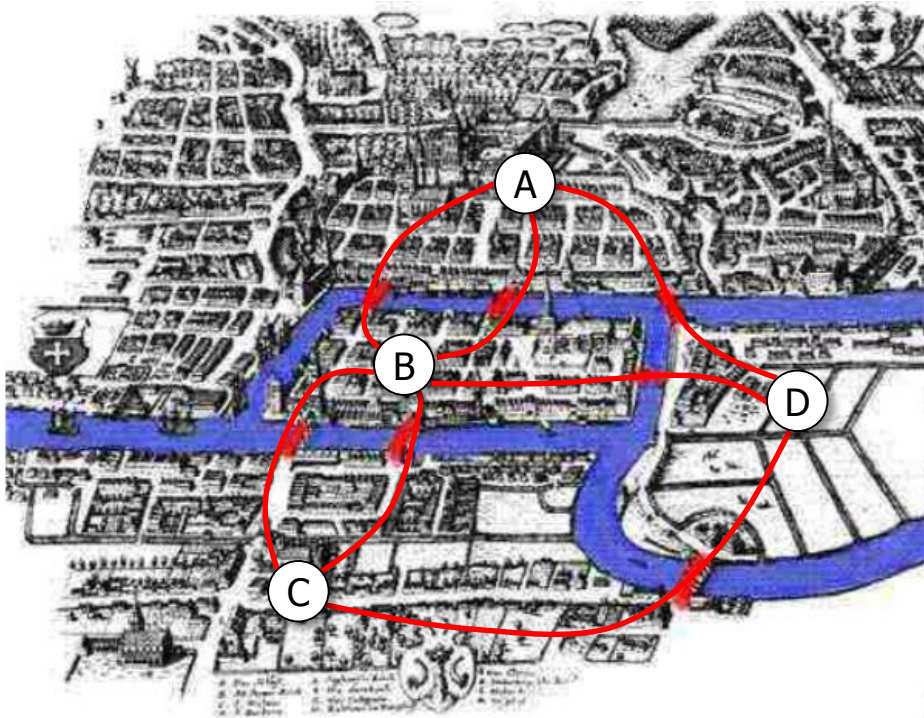


- **Problema:** é possível que uma pessoa faça um percurso na cidade de tal forma que inicie e volte para a mesma posição passando por todas as pontes uma única vez?

- A área de Teoria dos Grafos começa em 1736 quando Leonhard Euler publicou um artigo contendo a solução para o problema das sete pontes de Königsberg.
- Euler resolveu este problema transformando-o em um **grafo**.
 - ✓ Todos os “pontos” de uma dada área de terra podem ser representados por um único ponto, já que uma pessoa pode andar de um lado para o outro sem atravessar uma ponte.
 - ✓ Um ponto é conectado a outro se houver uma ponte de um lado para o outro.

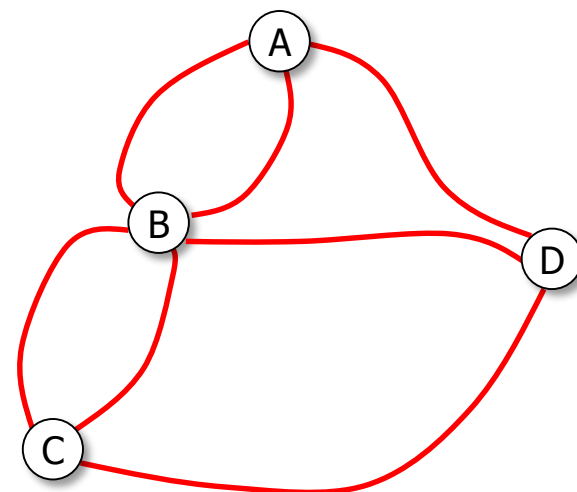


➤ Graficamente:



- **Problema:** é possível achar um caminho que comece e termine num vértice qualquer (A, B, C, ou D) e passe por cada aresta uma única vez?
- Aparentemente não!

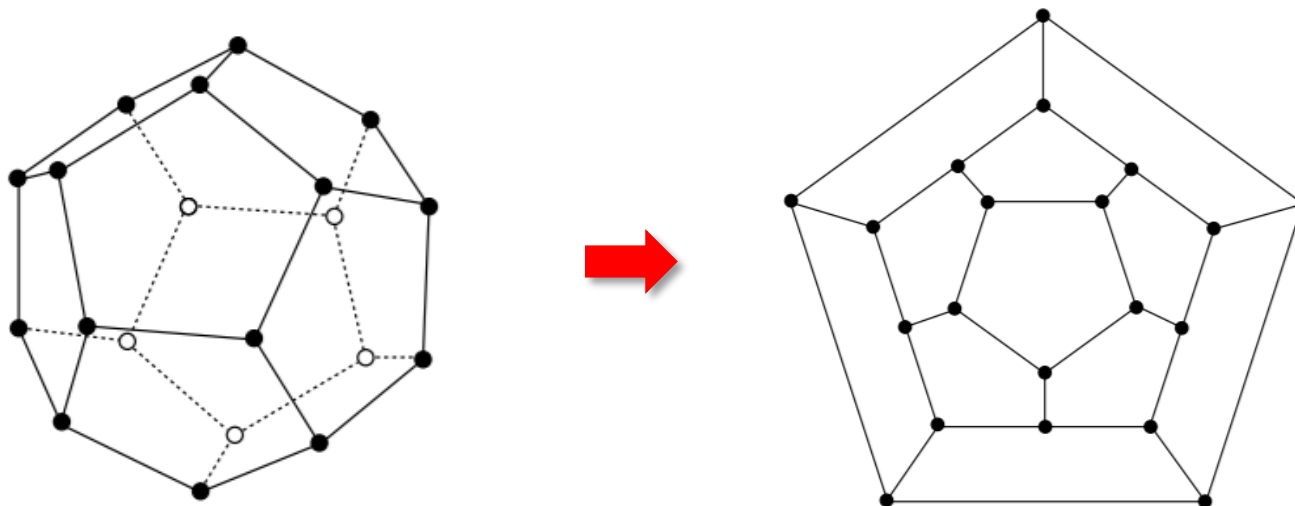
- ✓ Partindo do vértice A, toda vez que se passa por qualquer outro vértice, duas arestas são usadas: a de “chegada” e a de “saída”.
- ✓ Assim, se for possível achar uma rota que usa todas as arestas do grafo e começa e termina em A, o grau de cada vértice deve ser 2.



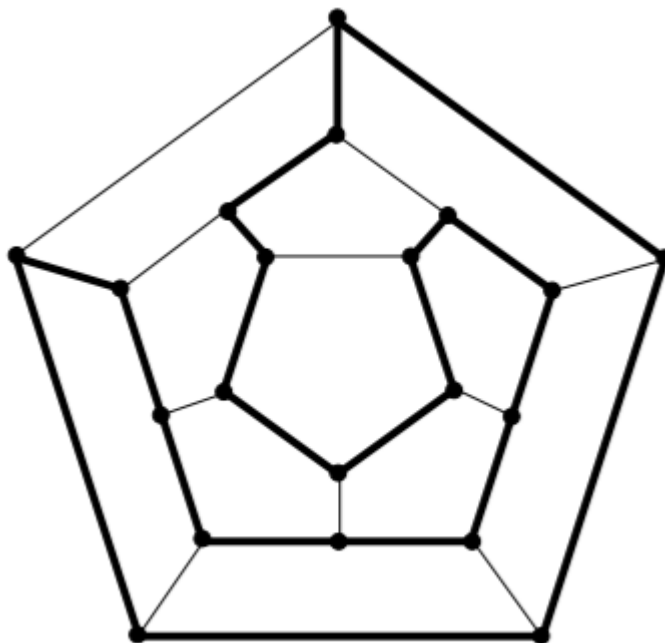
- **Definição:** dado um grafo $G(V, A)$, um circuito Euleriano para G é um **circuito** que contém **todos** os vértices $v \in V$ e **todas** arestas $a \in A$, de forma que ele passe **pelo menos uma vez** em cada vértice v e **uma única vez** em cada aresta a .
- **Teorema:** se o grafo $G(V, A)$ é conexo e para todo $v \in V$, $\text{grau}(v)$ é par, então G tem um circuito Euleriano.
- **Observação:** lembre-se que um circuito inicia e termina no mesmo vértice.

- **Definição:** dado um grafo $G(V, A)$ e vértices $v, w \in V$, um trajeto Euleriano de v para w é uma sequência de arestas e vértices adjacentes que começa em v , termina em w e passa por cada vértice de G pelo menos uma vez e passa por cada aresta de G exatamente uma única vez.
- **Teorema:** seja $G(V, A)$ um grafo e $v, w \in V$. Se G é conexo e $\text{grau}(v)$ é ímpar e $\text{grau}(w)$ é ímpar e o grau dos demais vértices é par, então G tem um trajeto Euleriano.
- **Observação:** lembre-se que um trajeto inicia e termina em vértices distintos.

- Em 1859, William Hamilton (matemático irlandês) propôs um jogo na forma de um dodecaedro (sólido de 12 faces), onde cada vértice representava uma cidade
- **Problema:** é possível iniciar em uma cidade e visitar todas as outras cidades exatamente uma única vez e retornar à cidade de partida?
- O jogo é mais fácil de ser visualizado projetando o dodecaedro no plano:



- **Definição:** dado um grafo $G(V, A)$, um caminho Hamiltoniano para G é um **caminho** que inclui **todos** os vértices de G .



- **Observação:** quando o caminho Hamiltoniano inicia e termina no mesmo vértice, ele é chamado de **ciclo Hamiltoniano**.

➤ Circuito Euleriano

- ✓ Inclui **todas** as arestas uma **única** vez.
- ✓ Inclui **todos** os vértices e pode **repeti-los**.
- ✓ Não gera, obrigatoriamente, um caminho Hamiltoniano, pois pode repetir os vértices.

➤ Caminho Hamiltoniano

- ✓ Inclui as arestas uma **única** vez, mas **não** precisa incluir todas.
- ✓ Inclui **todos** os vértices uma **única** vez (exceto o inicial e final).
- ✓ Não gera, obrigatoriamente, um circuito Euleriano, pois pode não incluir todas as arestas.

- Existem duas formas principais para representação do grafo $G(V, A)$:
 1. Matriz de Adjacências
 2. Lista de Adjacências

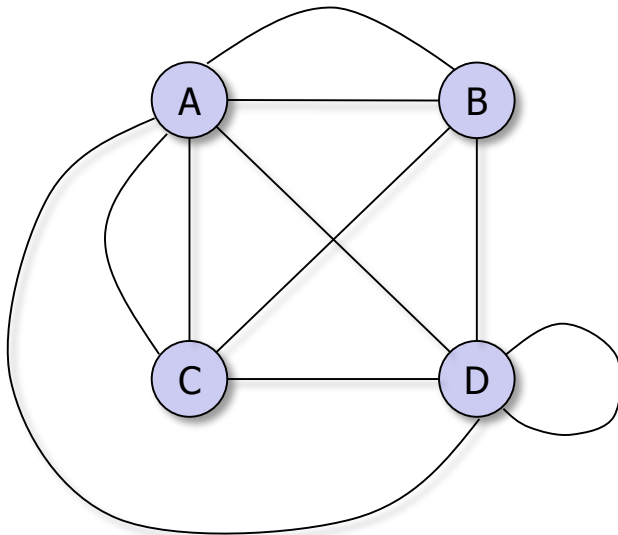
- Notação:
 - ✓ $|V|$ = número de vértices
 - ✓ $|A|$ = número de arestas

- O grafo é representado por uma matriz quadrada M de dimensões $n \times n$ onde $n = |A|$, ou seja, matriz tem $|A|^2$ elementos.
- O conteúdo da matriz M depende de G ser **orientado/não orientado** e **ponderado/não ponderado**.

	Não orientado	Orientado
Não ponderado	$M_{i,j} = \begin{cases} \text{arestas entre } i \text{ e } j, & \text{se } (i,j) \in A \text{ e } i \neq j \\ \text{no. laços,} & \text{se } (i,j) \in A \text{ e } i = j \\ 0, & \text{se } (i,j) \notin A \end{cases}$	$M_{i,j} = \begin{cases} \text{arestas de } i \rightarrow j, & \text{se } (i,j) \in A \\ 0, & \text{se } (i,j) \notin A \end{cases}$
Ponderado	$M_{i,j} = \begin{cases} \text{peso entre } i \text{ e } j, & \text{se } (i,j) \in A \\ 0, & \text{se } (i,j) \notin A \end{cases}$	$M_{i,j} = \begin{cases} \text{peso de } i \rightarrow j, & \text{se } (i,j) \in A \\ 0, & \text{se } (i,j) \notin A \end{cases}$

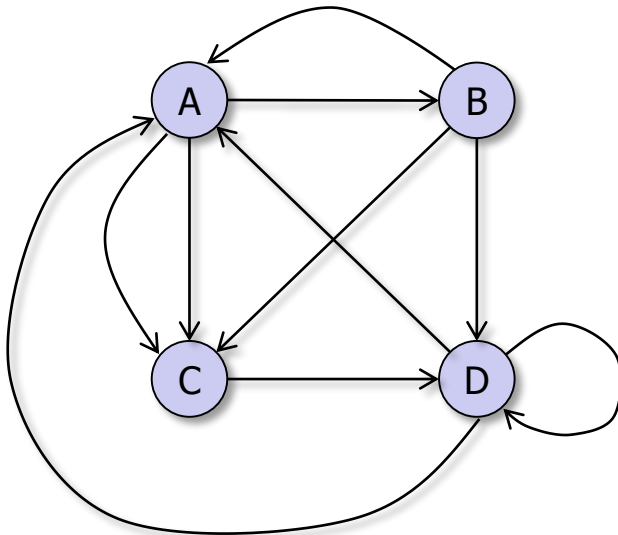
- **Atenção:** a matriz de adjacências não consegue representar multigrafos ponderados!

- Exemplo 1: grafo não orientado não ponderado



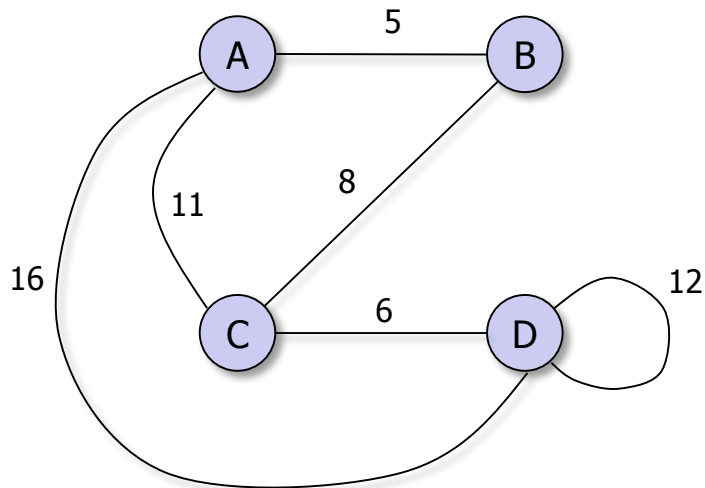
	A	B	C	D
A	0	2	2	2
B	2	0	1	1
C	2	1	0	1
D	2	1	1	1

- Exemplo 2: grafo orientado não ponderado



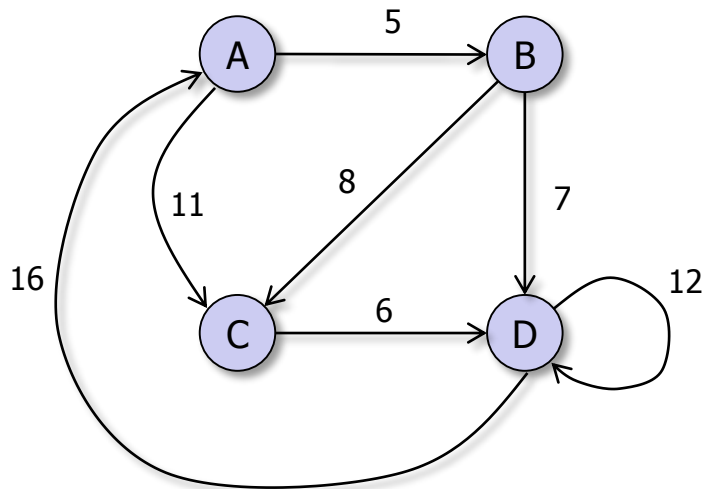
	A	B	C	D
A	0	1	2	0
B	1	0	1	1
C	0	0	0	1
D	2	0	0	1

➤ Exemplo 3: grafo não orientado ponderado



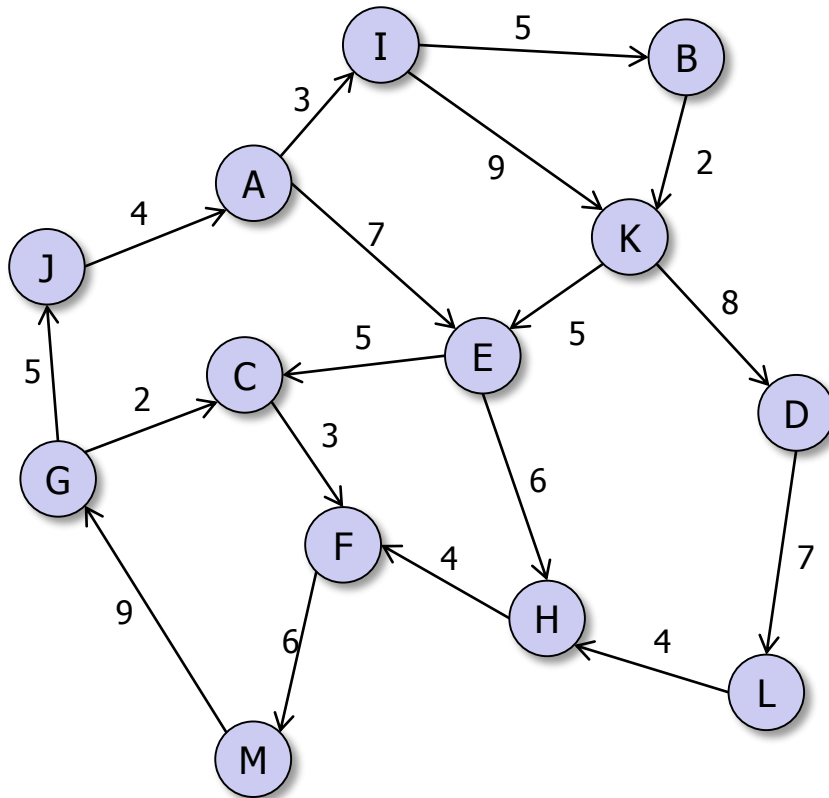
	A	B	C	D
A	0	5	11	16
B	5	0	8	0
C	11	8	0	6
D	16	0	6	12

➤ Exemplo 4: grafo orientado ponderado



	A	B	C	D
A	0	5	11	0
B	0	0	8	7
C	0	0	0	6
D	16	0	0	12

➤ Exemplo 5: grafo orientado ponderado



	A	B	C	D	E	F	G	H	I	J	K	L	M
A	0	0	0	0	7	0	0	0	3	0	0	0	0
B	0	0	0	0	0	0	0	0	0	0	2	0	0
C	0	0	0	0	0	3	0	0	0	0	0	0	0
D	0	0	0	0	0	0	0	0	0	0	0	7	0
E	0	0	5	0	0	0	0	6	0	0	0	0	0
F	0	0	0	0	0	0	0	0	0	0	0	0	6
G	0	0	2	0	0	0	0	0	0	5	0	0	0
H	0	0	0	0	0	4	0	0	0	0	0	0	0
I	0	5	0	0	0	0	0	0	0	0	9	0	0
J	4	0	0	0	0	0	0	0	0	0	0	0	0
K	0	0	0	0	5	0	0	0	0	0	0	0	0
L	0	0	0	0	0	0	0	4	0	0	0	0	0
M	0	0	0	0	0	0	9	0	0	0	0	0	0

➤ Vantagens:

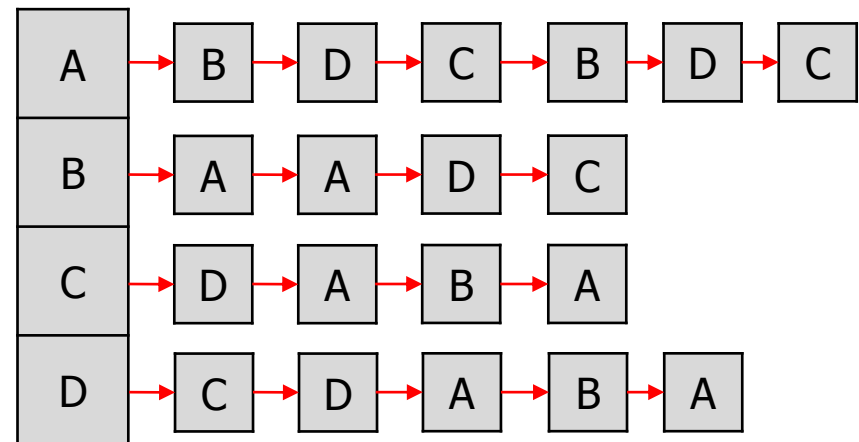
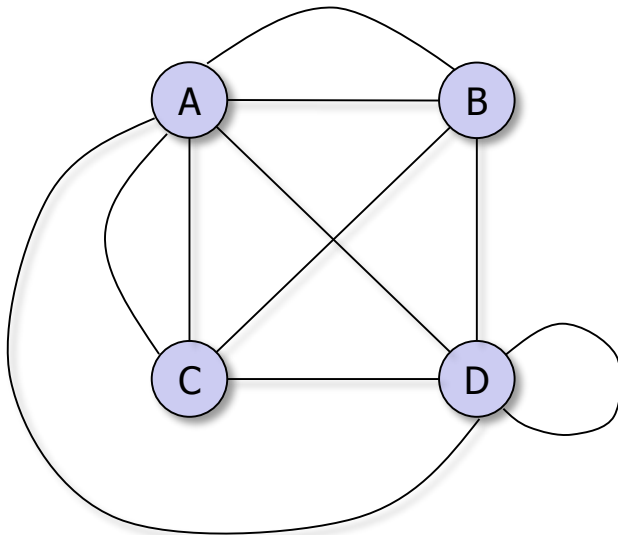
- ✓ Implementação simples e excelente desempenho (para saber se existe uma aresta ligando os vértices v e w basta consultar $M[i, j]$).
- ✓ Ideal para grafos **densos**, ou seja, grafos onde $|A| \cong |V|^2$.

➤ Desvantagens:

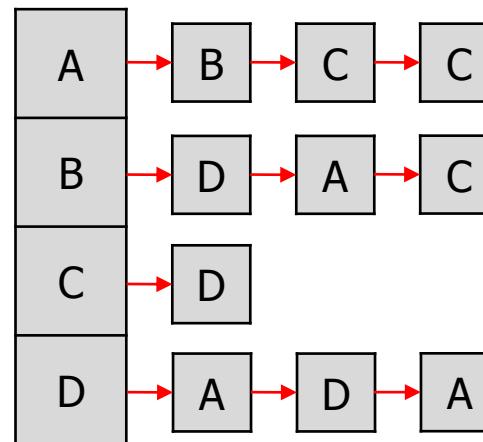
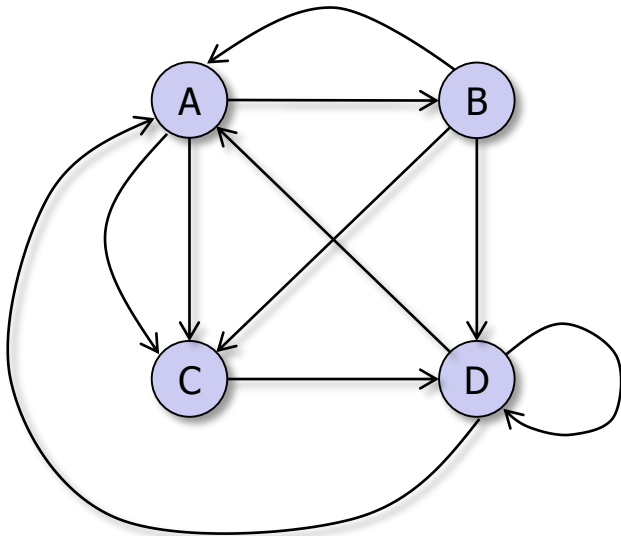
- ✓ Para os grafos **não orientados** há um grande desperdício de memória, já que a matriz é sempre **simétrica**.
- ✓ Para os grafos **esparsos** também há um grande desperdício de memória, pois a grande maioria dos valores da matriz é **zero**.

- O grafo é representado por um vetor de adjacências V de tamanho n , onde $n = |V|$.
- Para cada entrada V_i do vetor é construída uma **lista encadeada** com os vértices adjacentes à V_i .
- Não existe ordem para os vértices na lista encadeada.
- Caso o grafo seja ponderado, cada nó da lista armazena, além do vértice, o peso da respectiva aresta.

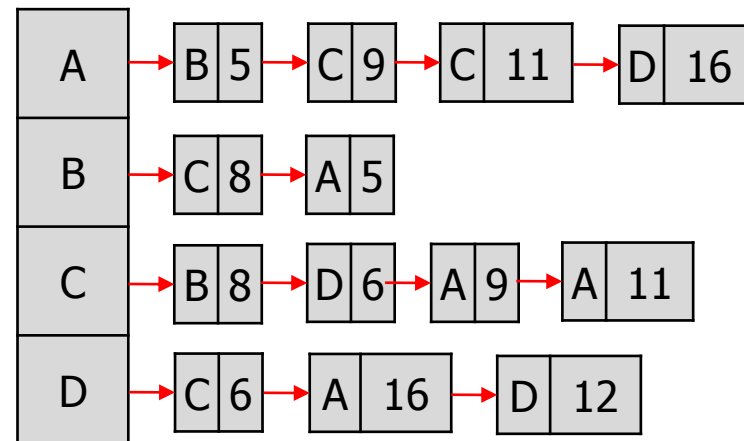
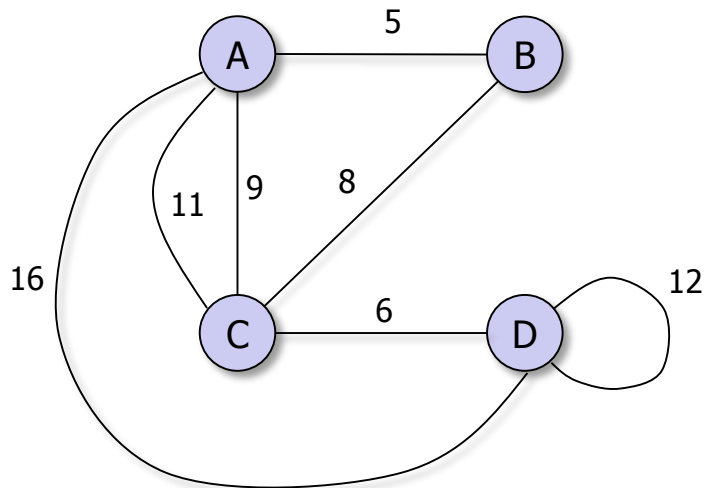
- Exemplo 1: grafo não orientado não ponderado



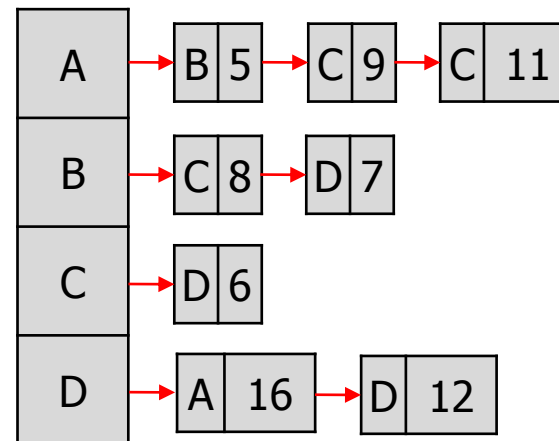
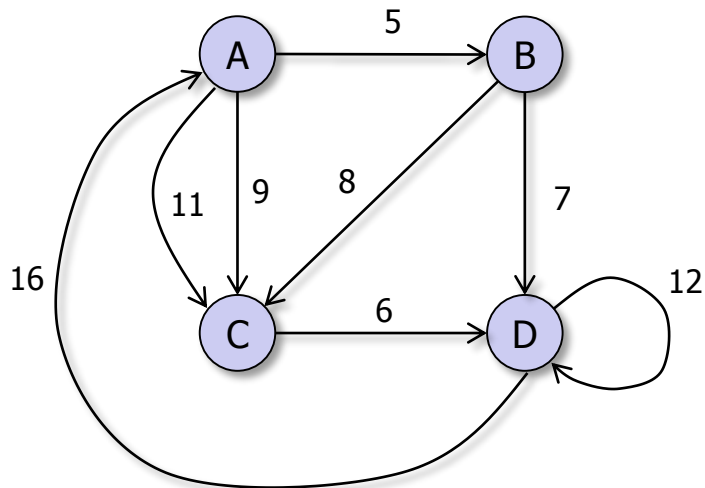
- Exemplo 2: grafo orientado não ponderado



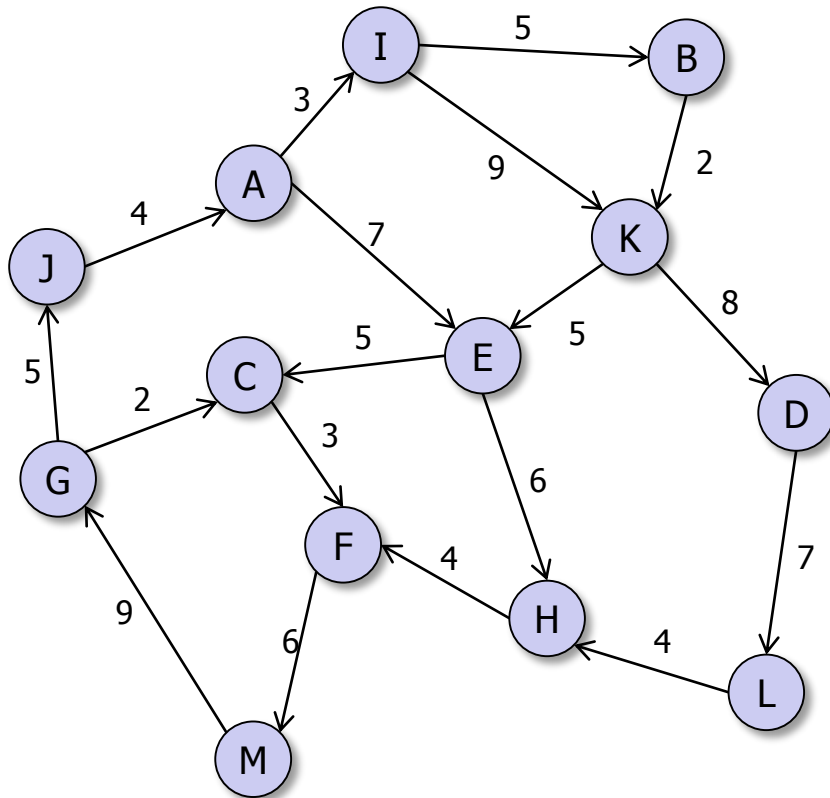
➤ Exemplo 3: grafo não orientado ponderado



➤ Exemplo 4: grafo orientado ponderado



➤ Exemplo 5: grafo orientado ponderado



A	→	I	3	→	E	7
B	→	K	2			
C	→	F	3			
D	→	L	7			
E	→	H	6			
F	→	M	6			
G	→	C	2	→	J	5
H	→	F	4			
I	→	B	5	→	K	9
J	→	A	4			
K	→	E	5			
L	→	H	4			
M	→	G	9			

➤ Vantagens:

- ✓ Representa **qualquer tipo** de grafo.
- ✓ Economia de memória para os casos de grafos **não orientados** e grafos **esparsos**.

➤ Desvantagens:

- ✓ Desempenho é pior, pois é necessário percorrer o vetor das arestas e a sua respectiva lista para encontrar determinada aresta.
- ✓ Não deve ser utilizado para grafos muito densos, pois nesse caso, ao invés de economizar demanda mais memória.

- Existem dois tipos de buscas comuns em grafos:
 1. Busca em Largura (Breadth-First Search – BFS)
 2. Busca em Profundidade (Depth-First Search – DFS)

- Característica: iniciando a busca a partir de um nó raiz, o algoritmo visita primeiro todos os nós adjacentes à raiz da busca (por níveis) e os nós adjacentes a cada um desses nós e assim por diante.
- É considerada, também, como uma busca por **nível**.
- Pode ser implementada utilizando-se uma **fila**.

BuscaLargura (v)

Marcar todos os vértices do grafo como NÃO PROCESSADO

fila $\leftarrow$ VAZIA

inserir(fila, v)

marcar(v, PROCESSADO)

enquanto fila não estiver vazia **faça**

 v $\leftarrow$ primeiro(fila)

 visitar(v)

para cada vértice w adjacente a v **faça**

 se w é NÃO PROCESSADO então

 inserir(fila, w)

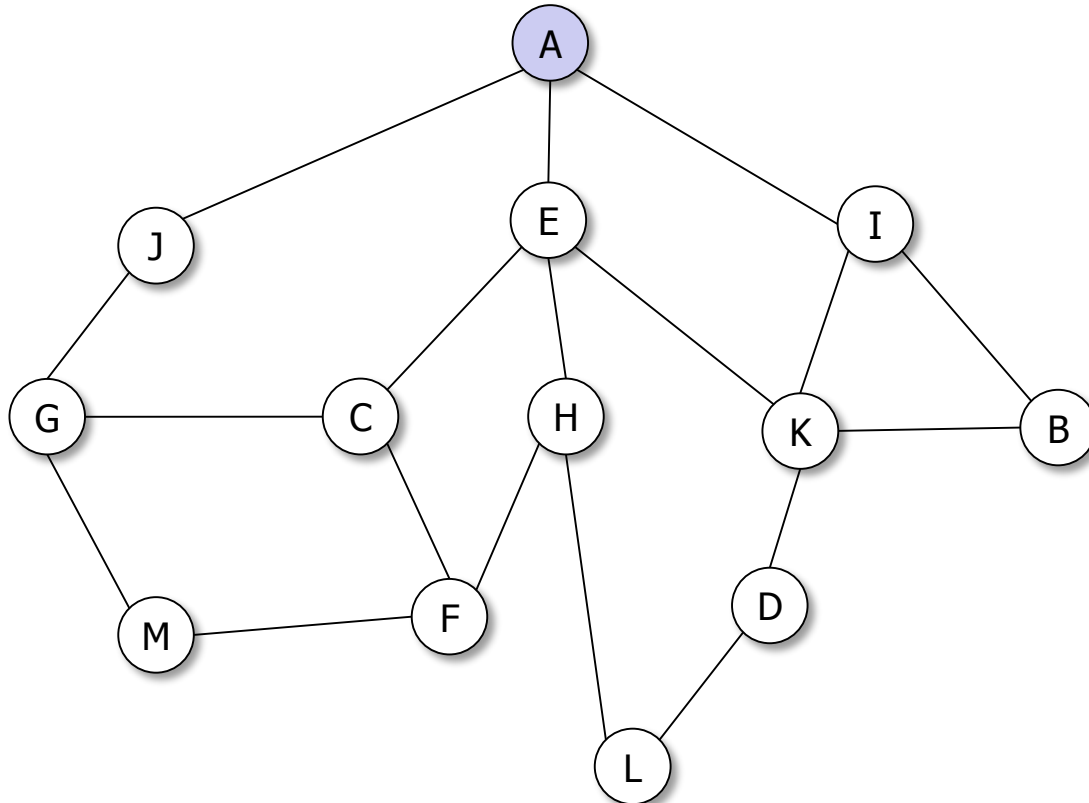
 marcar(w, PROCESSADO)

fim-se

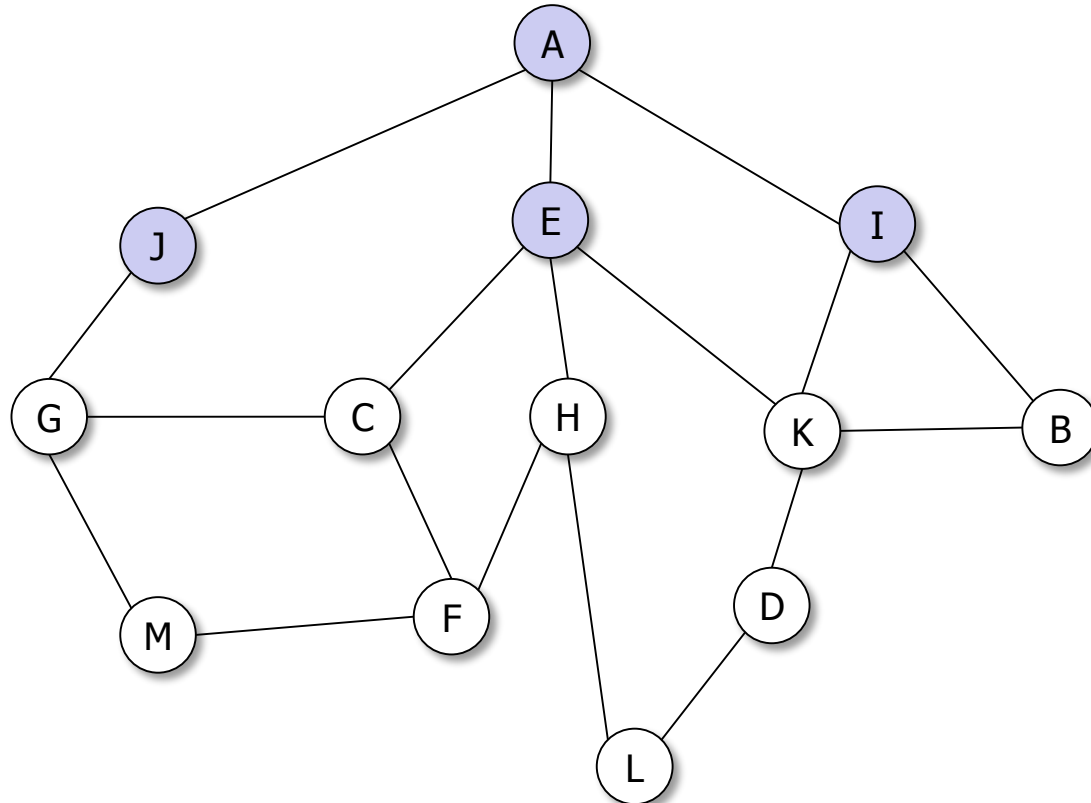
fim-para

fim-enquanto

➤ Fila: A

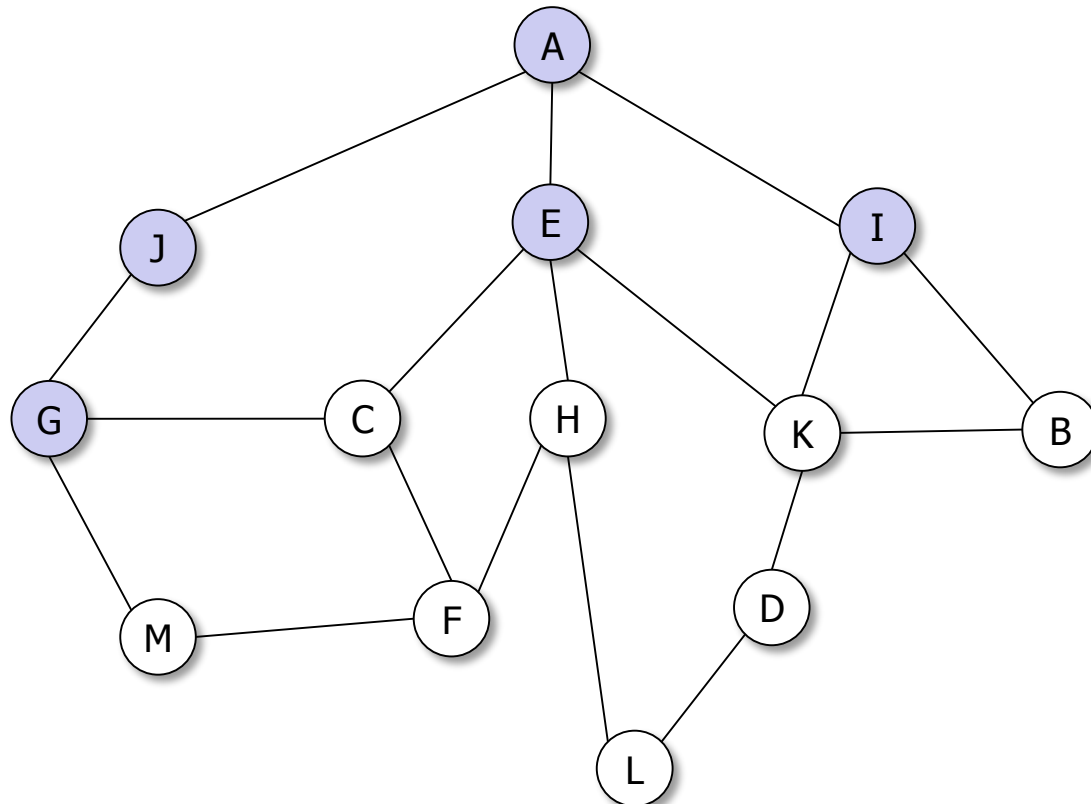


➤ Fila: J E I



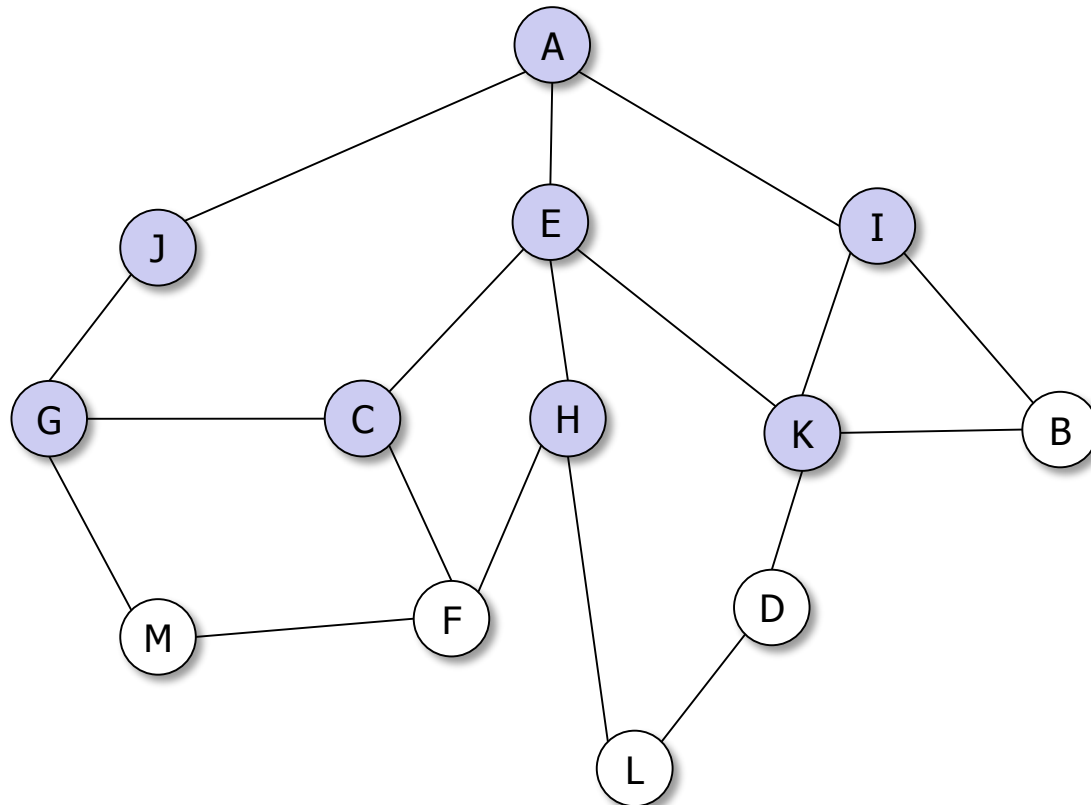
➤ A

➤ Fila: E I G



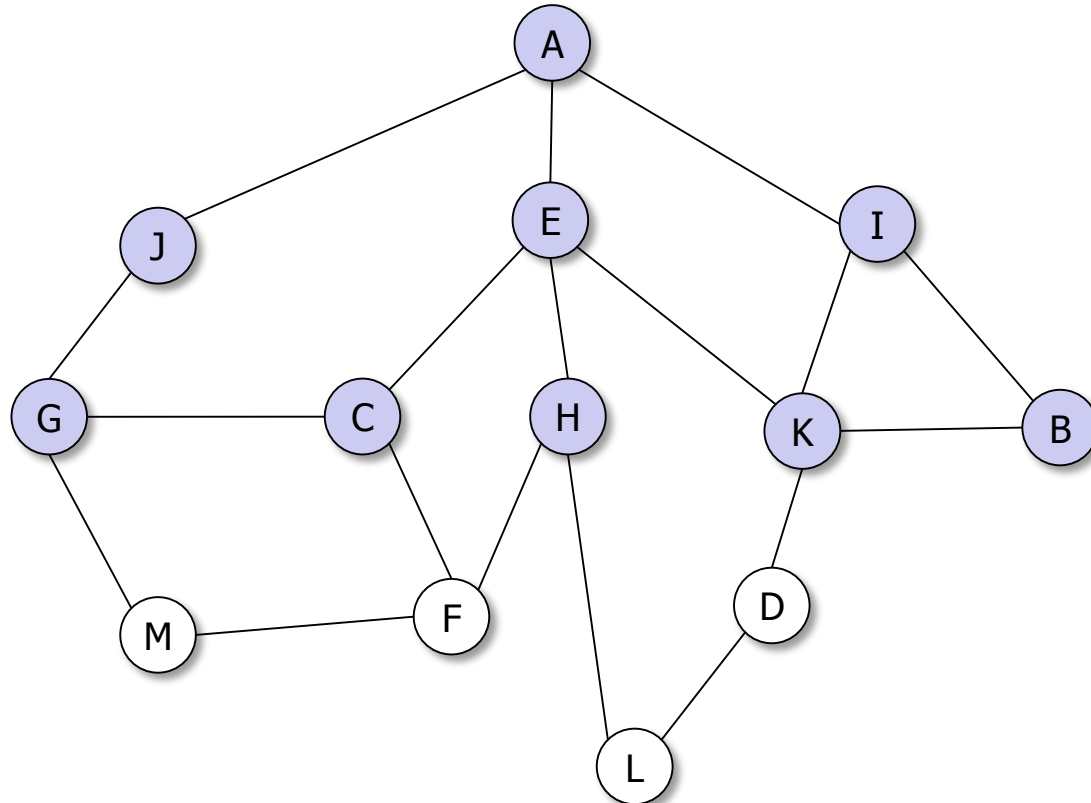
➤ A J

➤ Fila: I G C H K



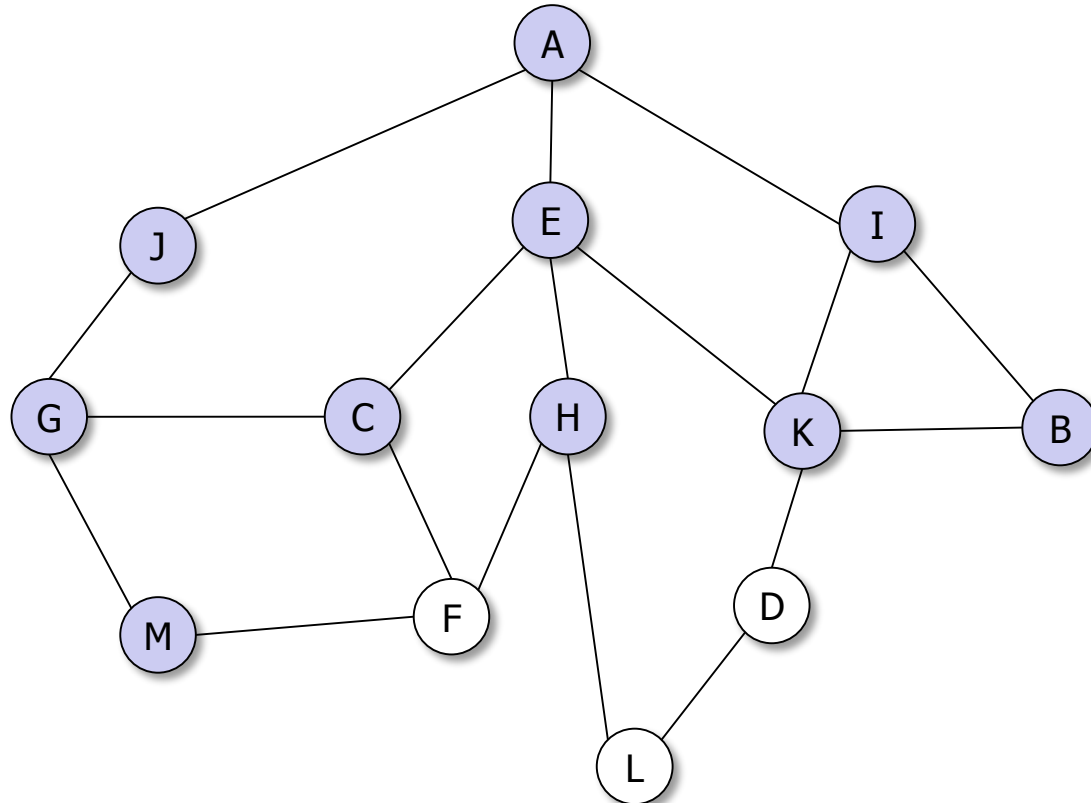
➤ A J E

➤ Fila: G C H K B



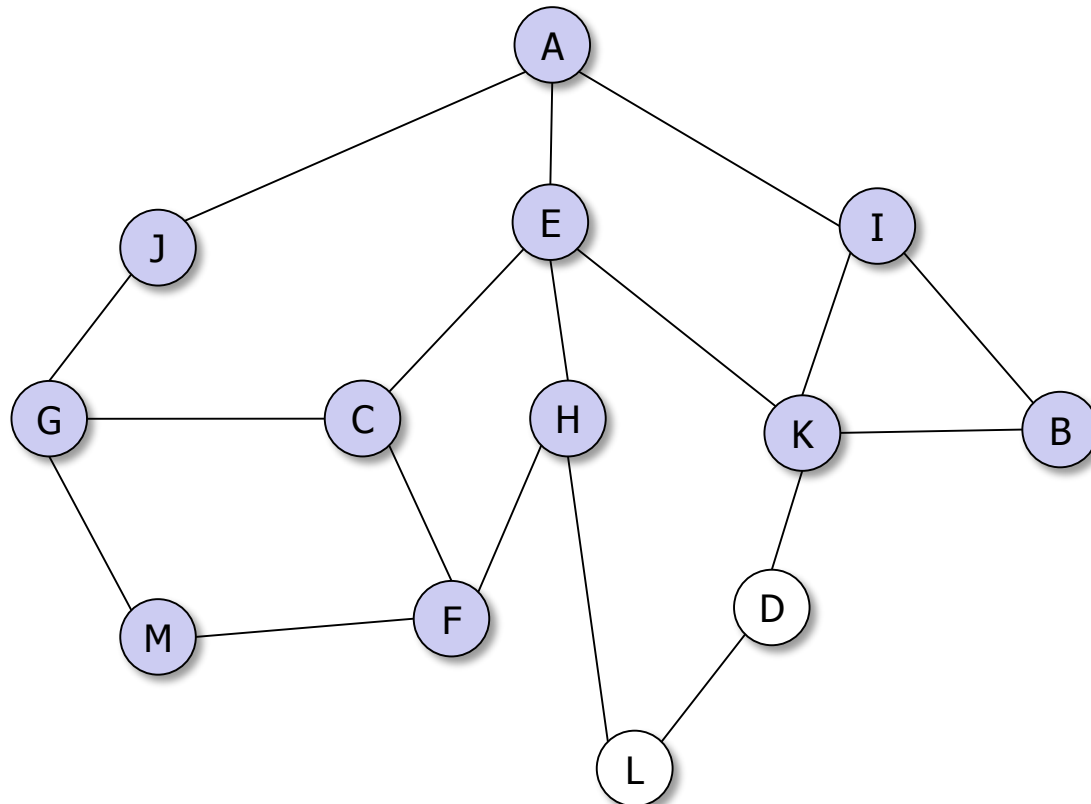
➤ A J E I

➤ Fila: C H K B M



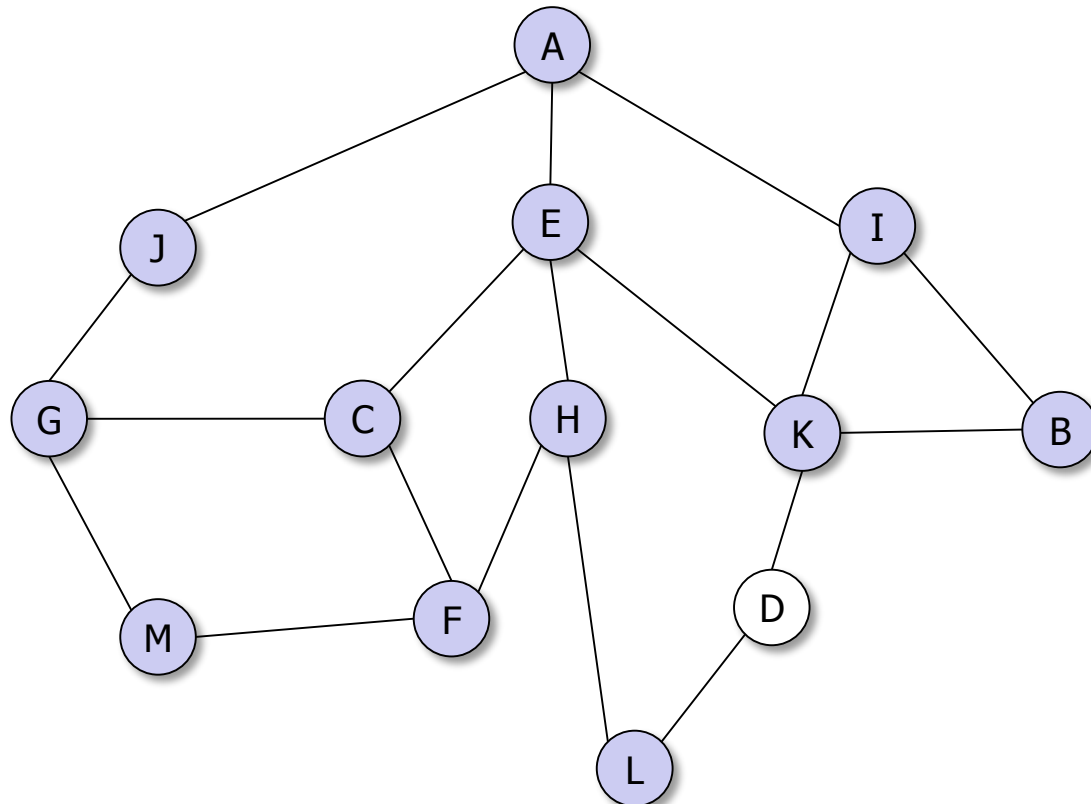
➤ A J E I G

➤ Fila: H K B M F



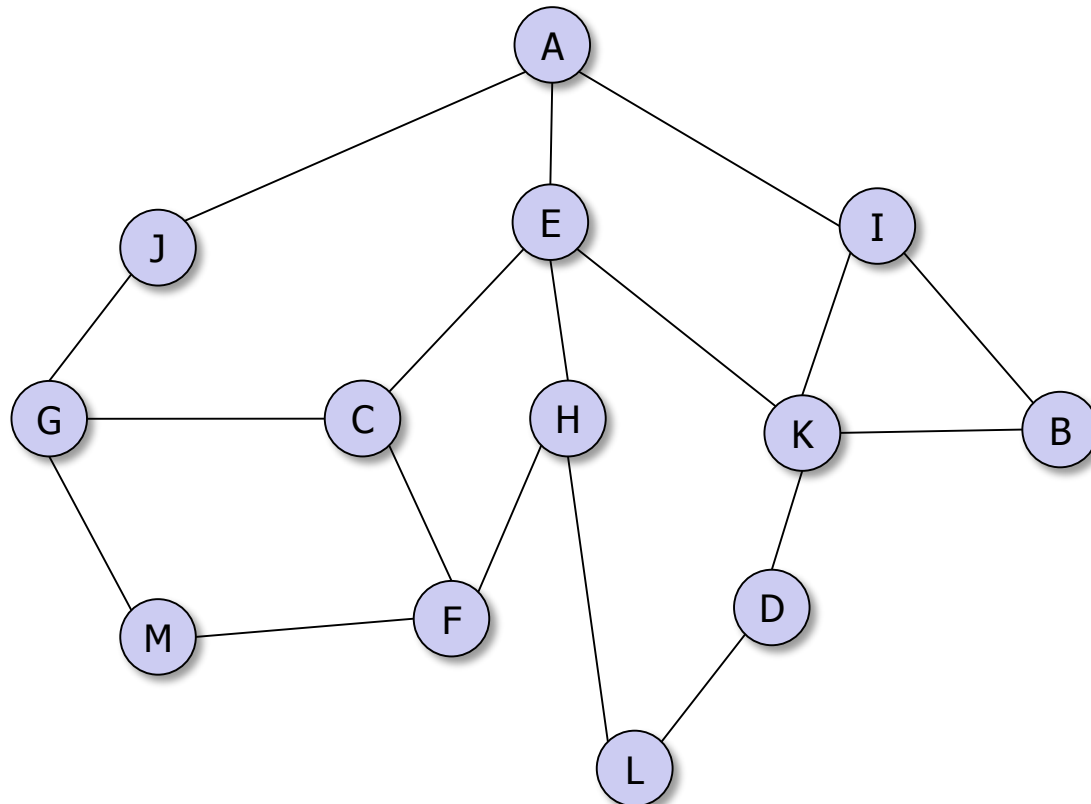
➤ A J E I G C

➤ Fila: K B M F L



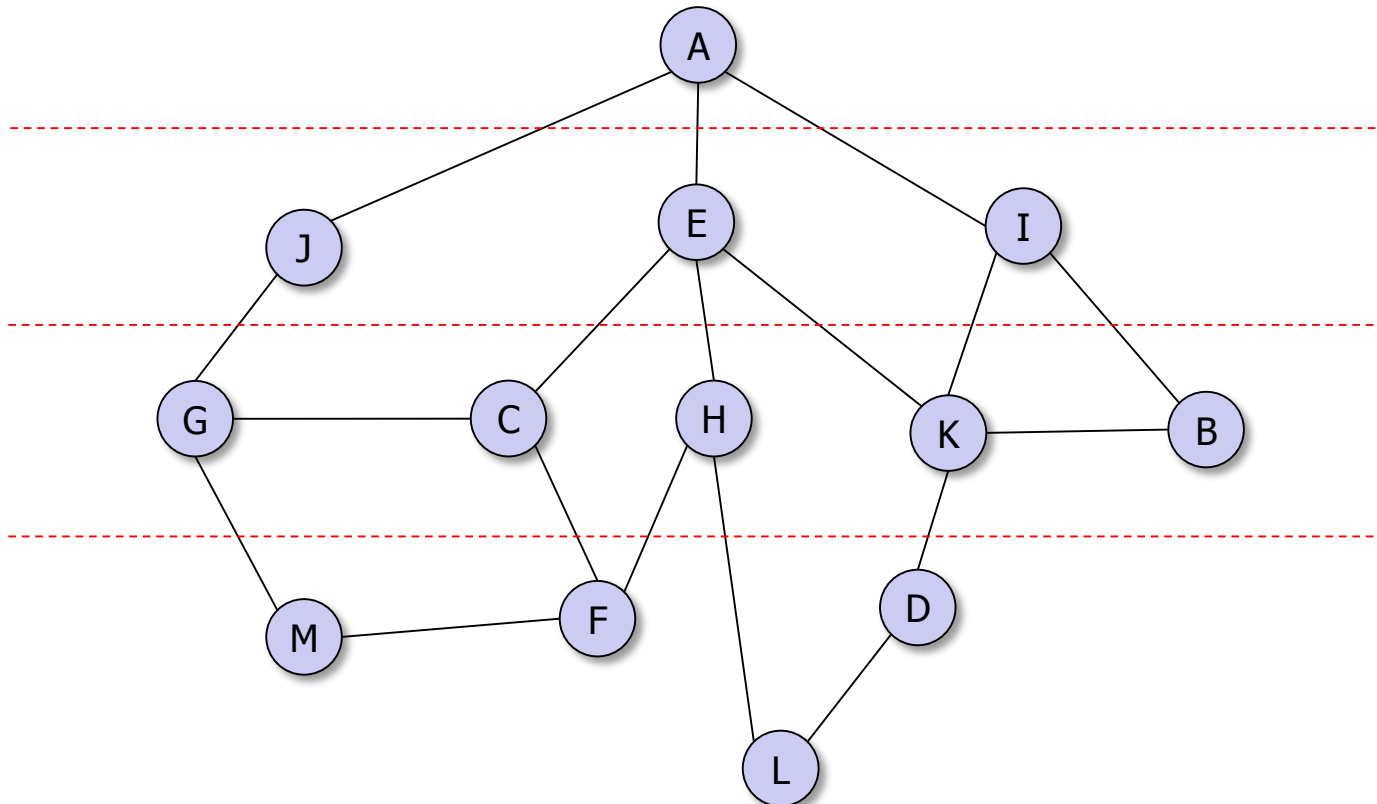
➤ A J E I G C H

➤ Fila: B M F L D



➤ A J E I G C H K

➤ Fila: vazia



➤ A J E I G C H K B M F L D

- Característica: iniciando a busca a partir de um nó raiz, o algoritmo visita um nó adjacente à raiz e, em seguida, outro nó adjacente a este e assim por diante, indo o mais profundo possível antes de retroceder e visitar outros nós.
- Pode ser implementada utilizando-se uma pilha.

BuscaProfundidade (v)

Marcar todos os vértices do grafo como NÃO PROCESSADO

pilha $\leftarrow$ VAZIA

empilhar(pilha, v)

marcar(v, PROCESSADO)

enquanto pilha não estiver vazia **faça**

 v $\leftarrow$ topo(pilha)

 visitar(v)

para cada vértice w adjacente a v **faça**

 se w é NÃO PROCESSADO então

 empilhar(pilha, w)

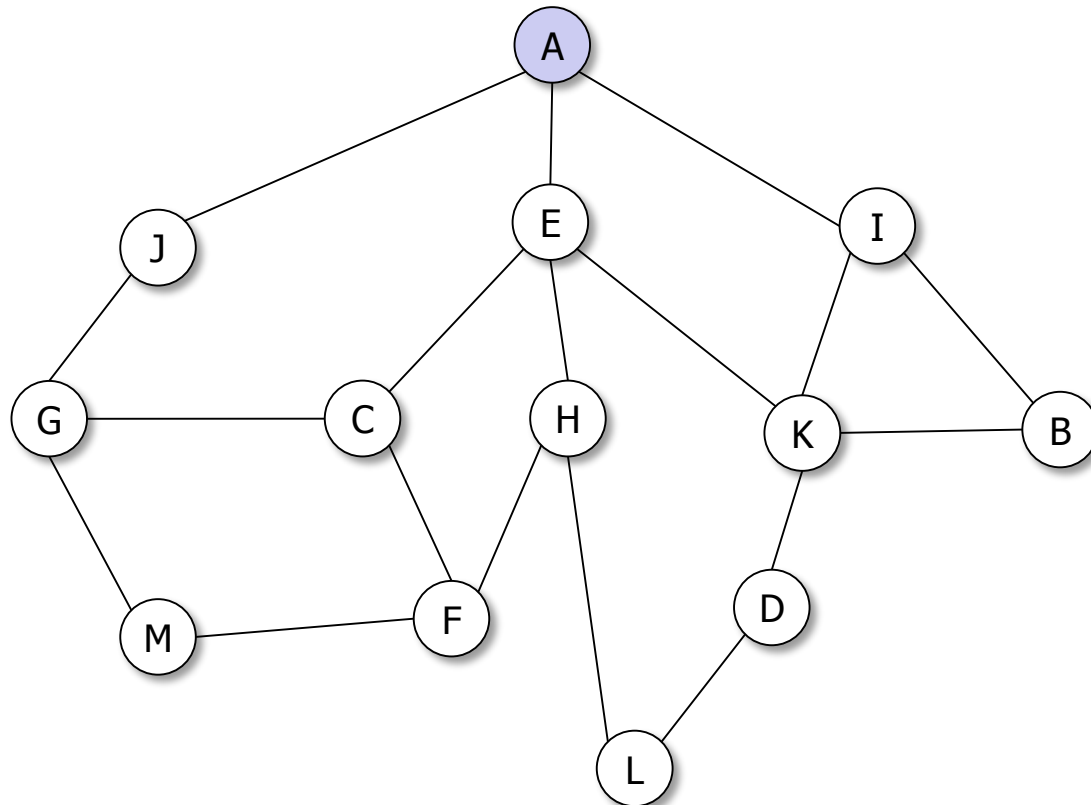
 marcar(w, PROCESSADO)

fim-se

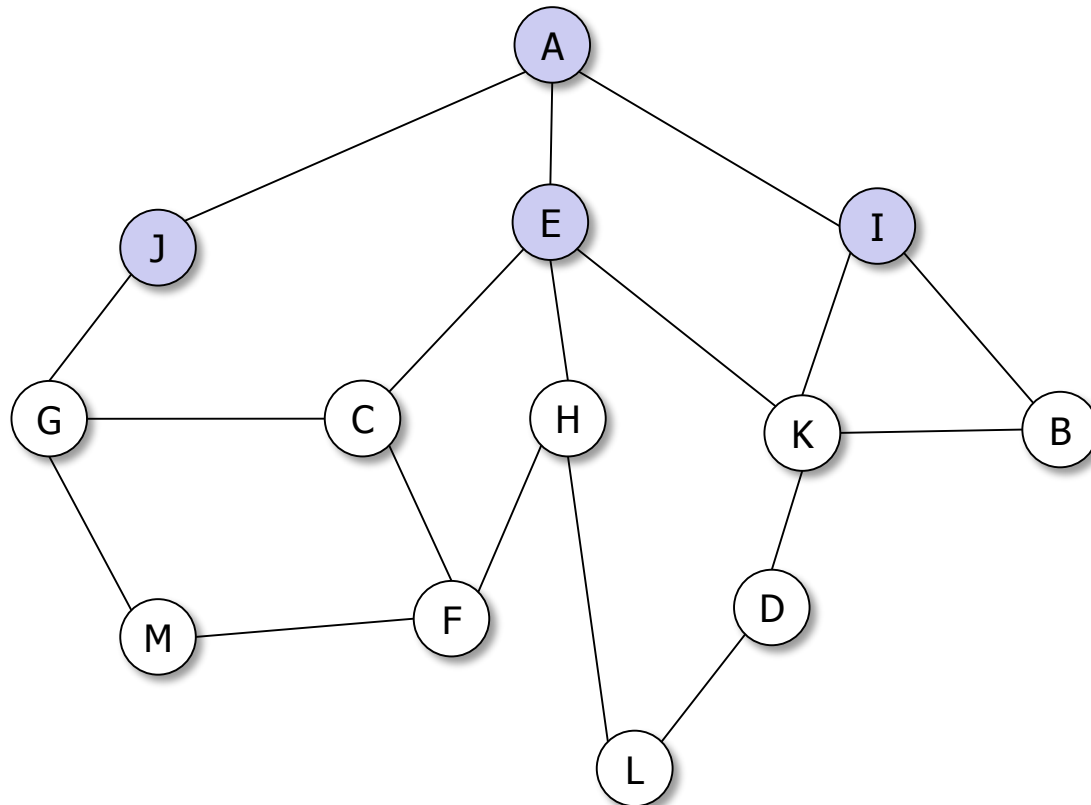
fim-para

fim-enquanto

➤ Pilha: A

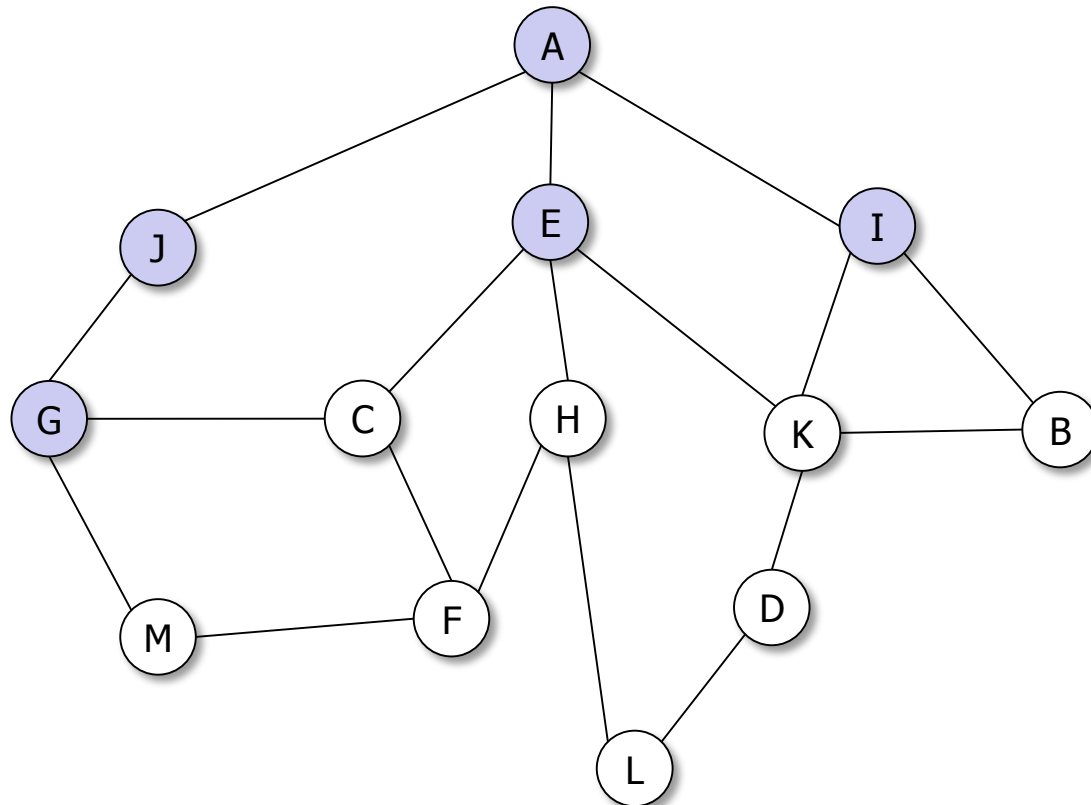


➤ Pilha: J
E
I



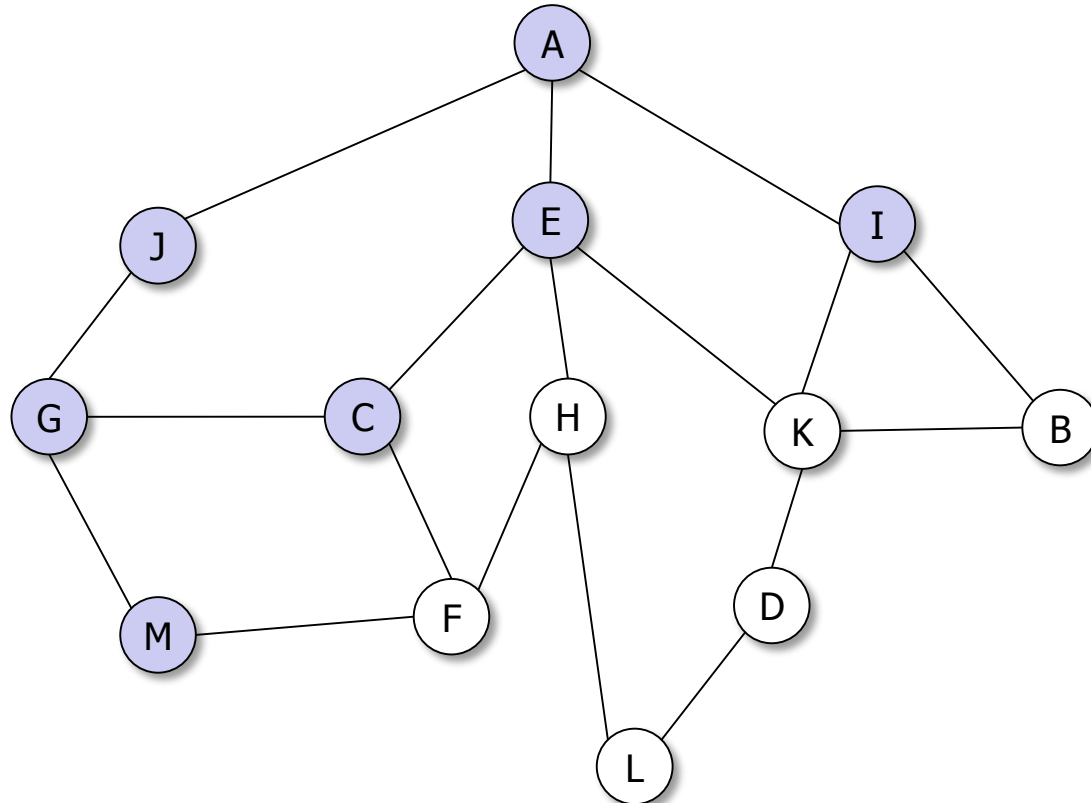
➤ A

➤ Pilha: G
E
I



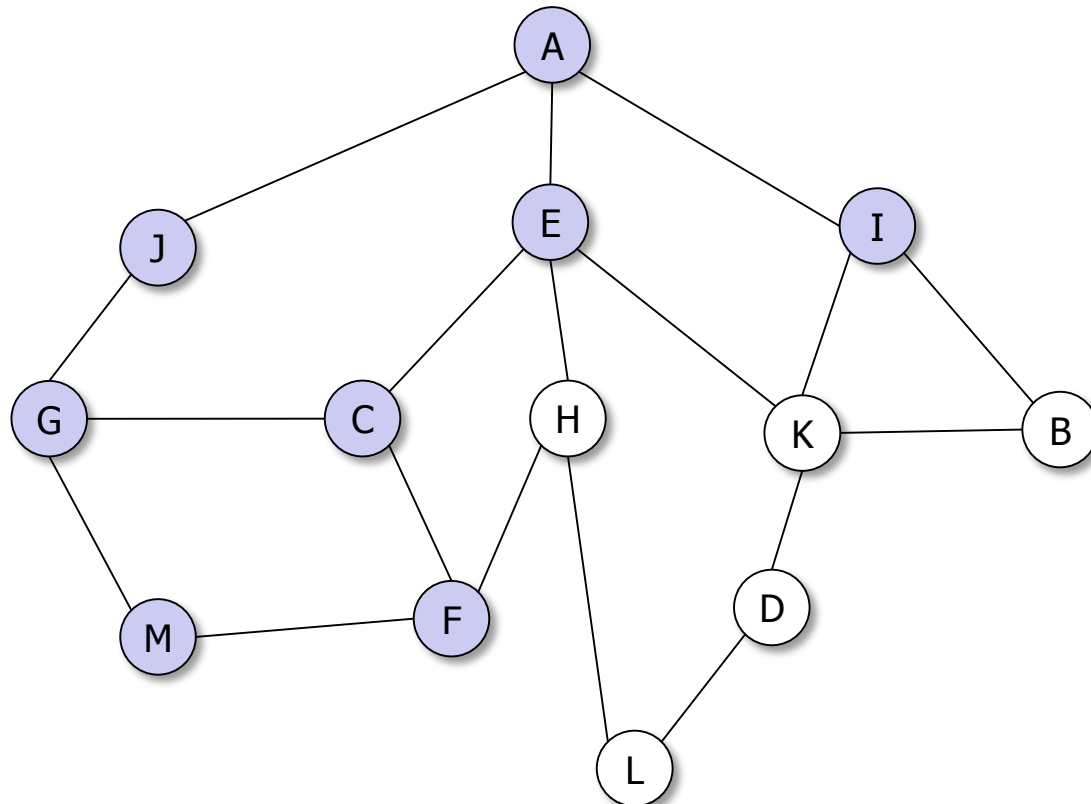
➤ A J

➤ Pilha: M
C
E
I



➤ A J G

➤ Pilha: F
C
E
I

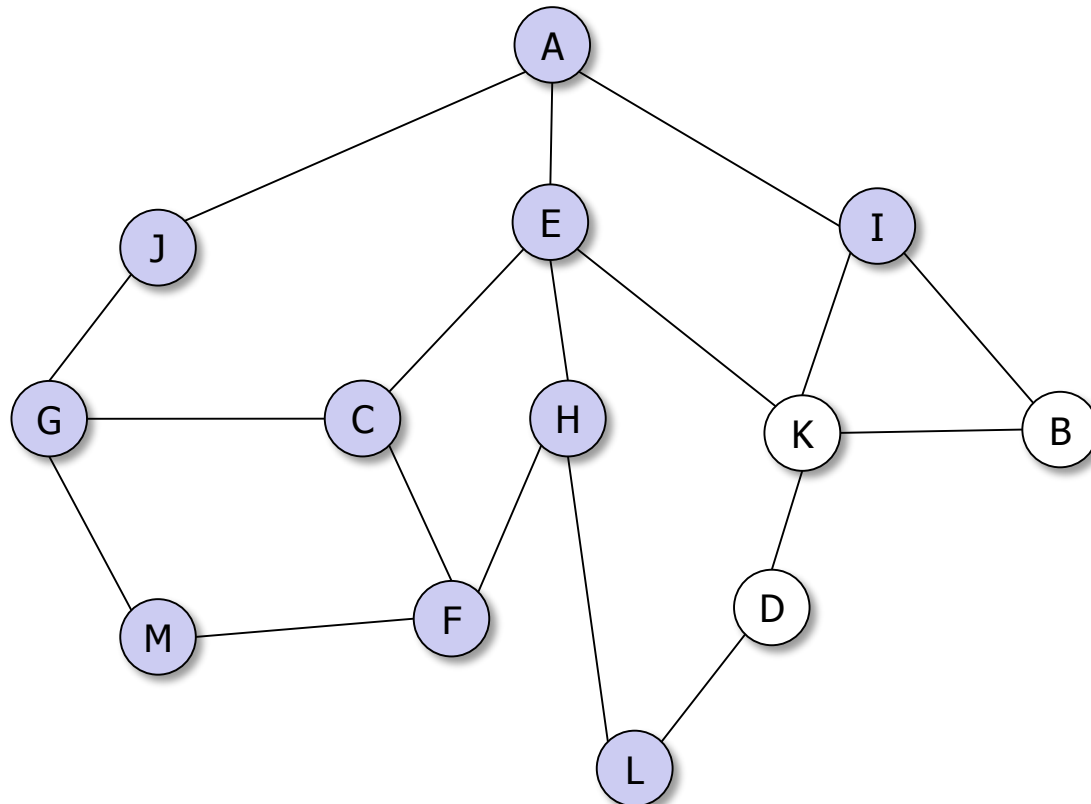


➤ A J G M

The graph consists of 12 nodes labeled A through L. The nodes are arranged in a hierarchical structure. Node A is at the top. It is connected to nodes J, E, and I. Node J is connected to nodes G and C. Node E is connected to nodes C, H, and K. Node I is connected to nodes K and B. Node G is connected to nodes C and M. Node C is connected to nodes F and H. Node H is connected to nodes F and L. Node K is connected to nodes D and B. Node D is connected to node L. Node M is connected to node F. Node L is connected to node D. The nodes are shaded light blue if they are part of a specific set, and white otherwise.

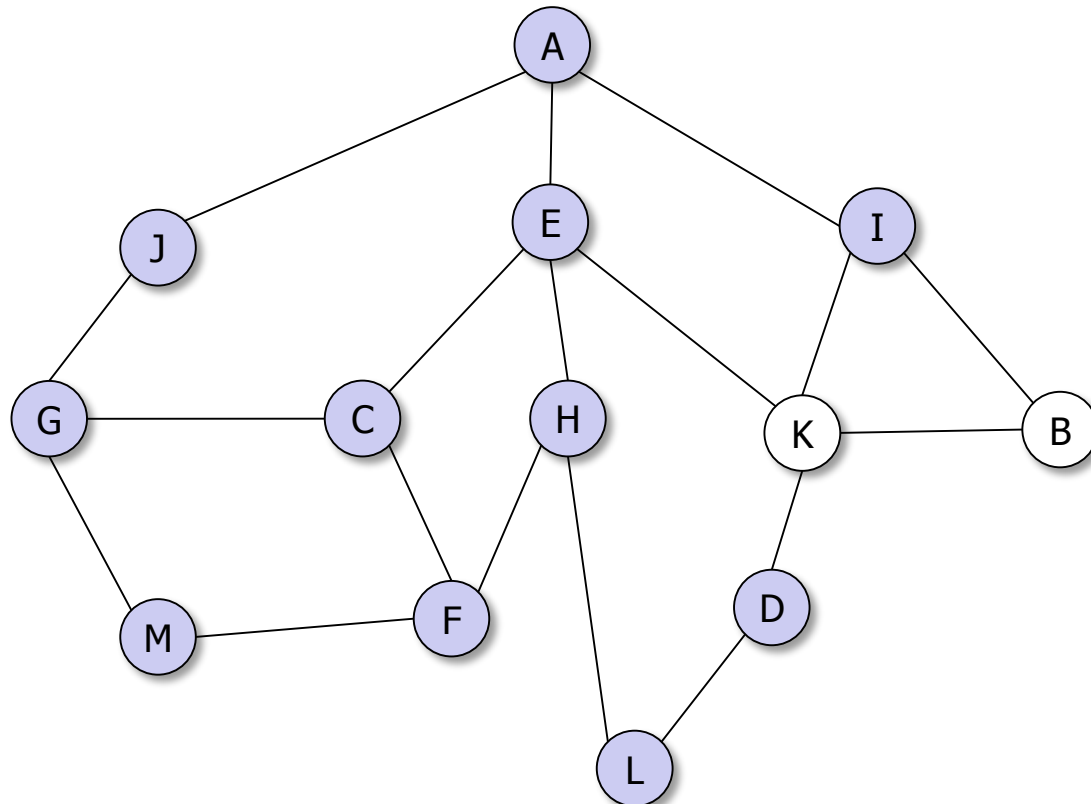
Estrutura de Dados II

➤ Pilha: L
C
E
I



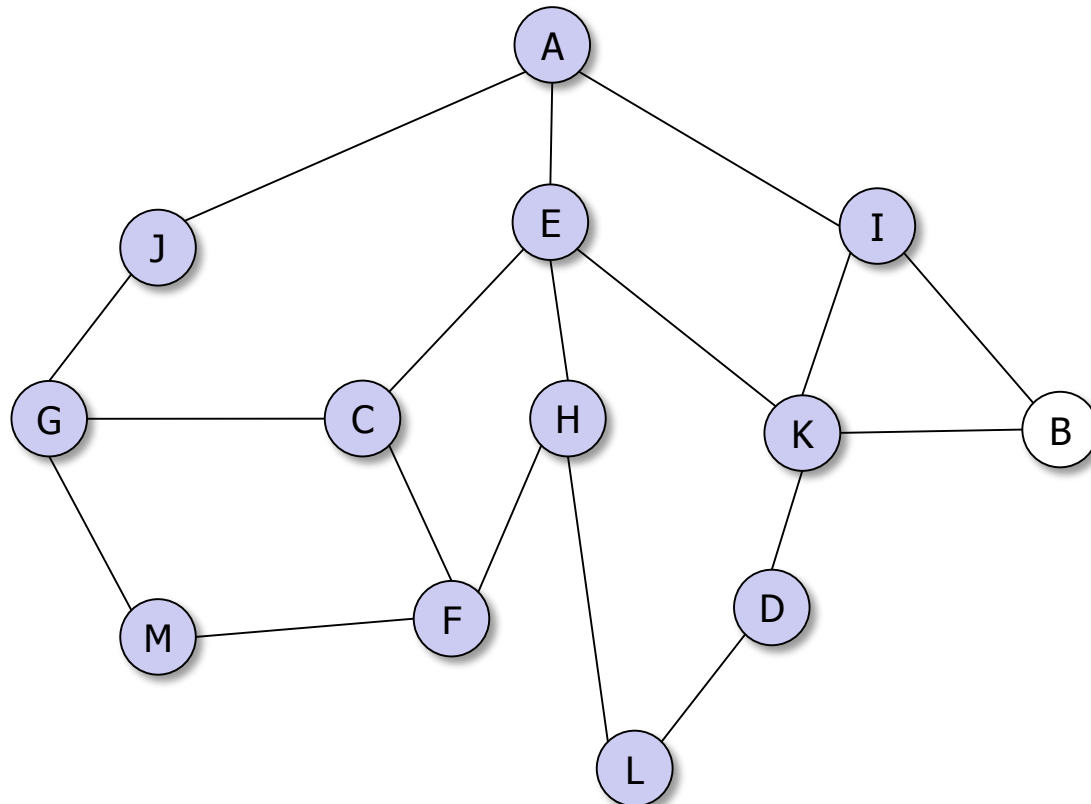
➤ A J G M F H

➤ Pilha: D
C
E
I



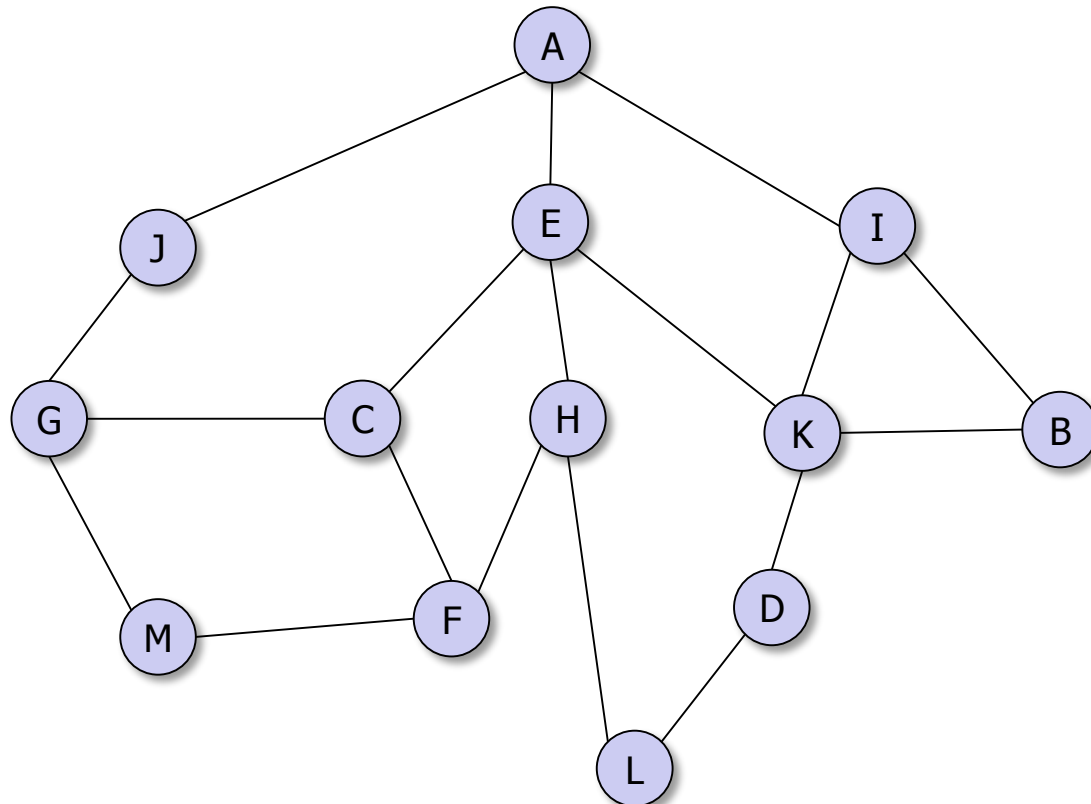
➤ A J G M F H L

➤ Pilha: K
C
E
I



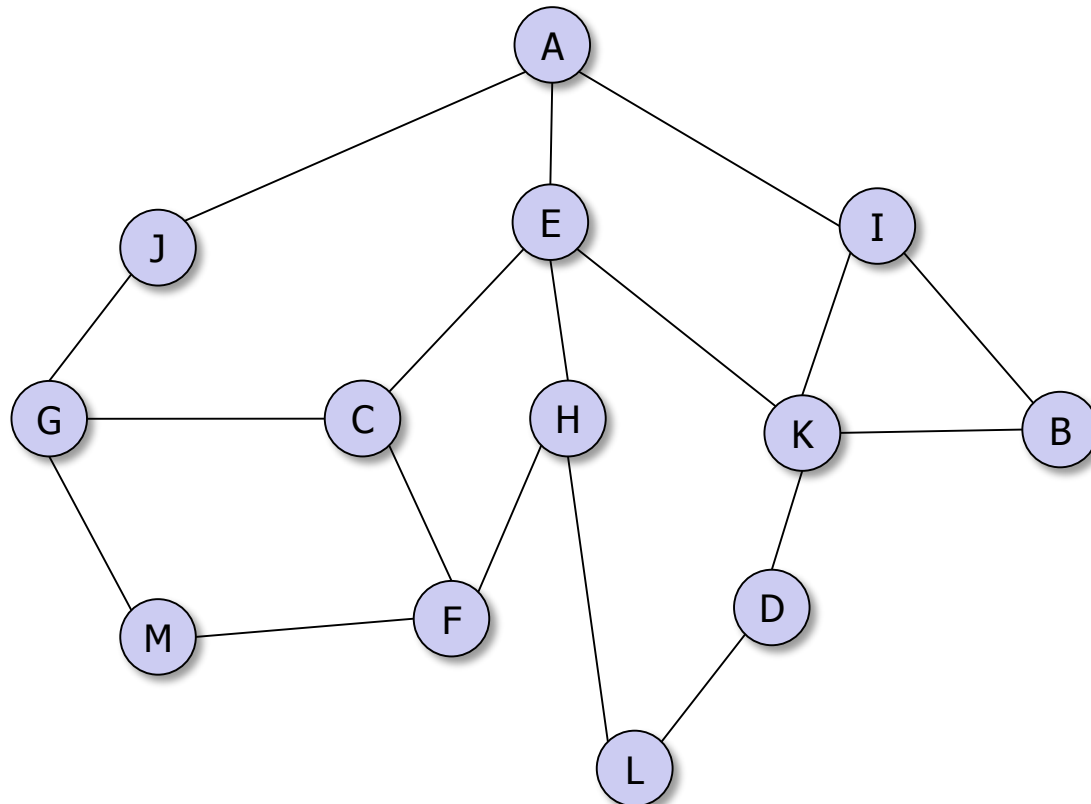
➤ A J G M F H L D

➤ Pilha: B
C
E
I



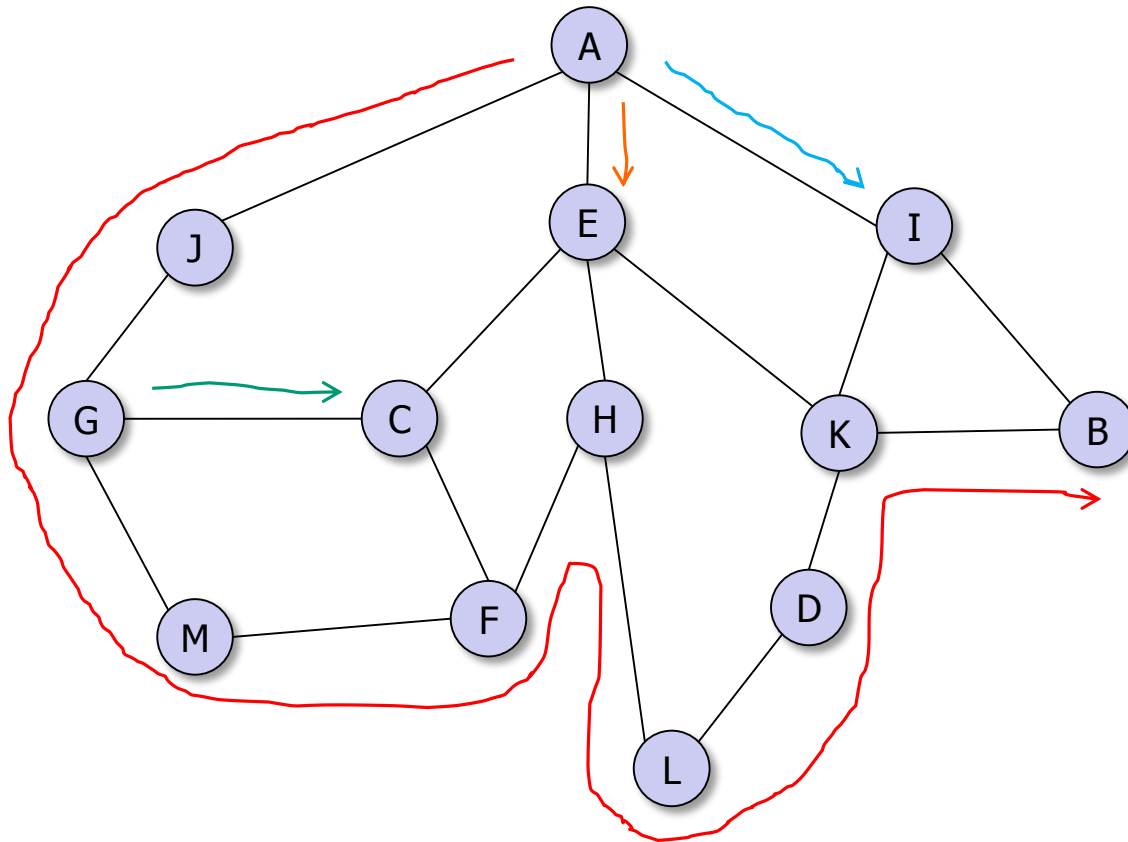
➤ A J G M F H L D K

➤ Pilha: C
E
I



➤ A J G M F H L D K B

➤ Pilha: vazia



➤ A J G M F H L D K B C E I

```
typedef struct grafo Grafo;

// Cria um Grafo com espaço para o número de vértices especificado. O grafo
// pode ser orientado ou não e ponderado ou não.
Grafo criaGrafo(int numVertices, bool orientado, bool ponderado);

// Cria um vértice com o rótulo especificado.
// Se já existir um vértice com esse rótulo retorna FALSO.
bool criaVertice(Grafo &grafo, char *rotulo);

// Remove o vértice com o rótulo especificado.
// Se não existir um vértice com esse rótulo retorna FALSO.
bool removeVertice(Grafo &grafo, char *rotulo);

// Cria uma aresta ligando dois vértices em um grafo ponderado. Se um dos
// vértices não existir ou o grafo for não ponderado retorna FALSO.
bool criaAresta(Grafo &grafo, char *origem, char *destino, int peso);

// Cria uma aresta ligando dois vértices em um grafo não ponderado. Se um
// dos vértices não existir ou o grafo for ponderado retorna FALSO.
bool criaAresta(Grafo &grafo, char *origem, char *destino);
```

```
// Remove a aresta que liga os dois vértices especificados. Se um
// dos vértices não existir retorna FALSO.
bool removeAresta(Grafo &grafo, char *origem, char *destino);

// Verifica se existe um vértice com o rótulo especificado.
bool existeVertice(Grafo &grafo, char *rotulo);

// Verifica se existe uma aresta ligando os vértices especificados.
bool existeAresta(Grafo &grafo, char *origem, char *destino);

// Retorna o peso da aresta que liga os vértices especificados.
// Retorna 0 se não existe a aresta ou se um dos vértices não existir.
int pesoAresta(Grafo &grafo, char *origem, char *destino);

// Percorre o grafo usando DFS a partir do vértice especificado.
void buscaProfundidade(Grafo &grafo, char *origem);

// Percorre o grafo usando BFS a partir do vértice especificado.
void buscaLargura(Grafo &grafo, char *origem);

// Libera a área de memória usada pelo grafo.
void libera(Grafo &grafo);
```

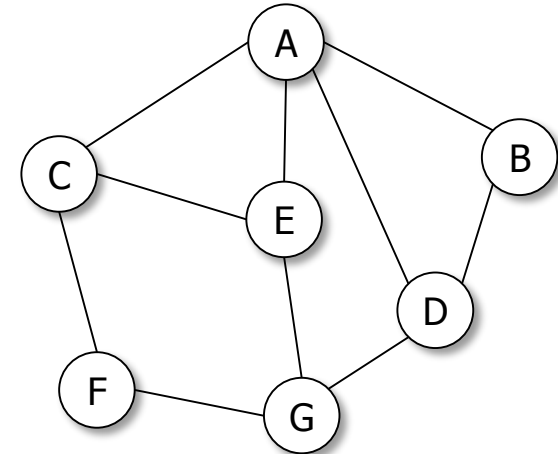
- A estrutura principal que implementa o grafo armazena informações sobre o tipo do grafo e seu tamanho, mais um ponteiro para o vetor de rótulos dos vértices e outro ponteiro para a matriz de adjacências:

```
struct grafo {  
    bool orientado;           // indicador de grafo orientado  
    bool ponderado;          // indicador de grafo ponderado  
    int maxVertices;          // número máximo de vértices  
    int qtdVertices;          // número atual de vértices  
    Vertice *vertices;        // ponteiro para o vetor com  
                                // os rótulos dos vértices  
    int **matriz;             // ponteiro para a matriz de  
                                // adjacências  
};  
  
typedef struct grafo Grafo;
```

➤ Exemplo 1:

orientado	F
ponderado	F
maxVertices	8
qtdVertices	7
vertices	
matriz	

0	A	F
1	B	F
2	C	F
3	D	F
4	E	F
5	F	F
6	G	F
7		

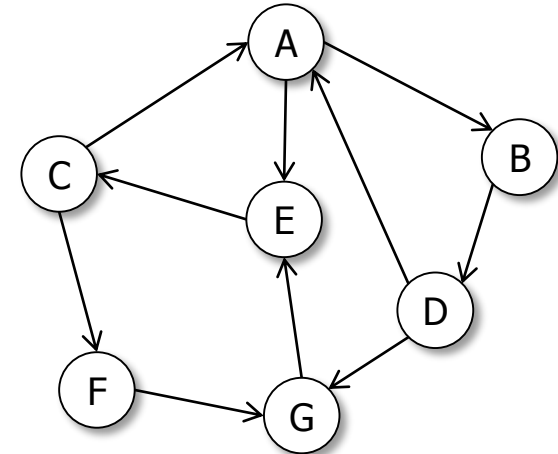


	0	1	2	3	4	5	6	7
0	0	1	1	1	1	0	0	0
1	1	0	0	1	0	0	0	0
2	1	0	0	0	1	1	0	0
3	1	1	0	0	0	0	1	0
4	1	0	1	0	0	0	1	0
5	0	0	1	0	0	0	1	0
6	0	0	0	1	1	1	0	0
7	0	0	0	0	0	0	0	0

➤ Exemplo 2:

orientado	T
ponderado	F
maxVertices	8
qtdVertices	7
vertices	
matriz	

0	A	F
1	B	F
2	C	F
3	D	F
4	E	F
5	F	F
6	G	F
7		

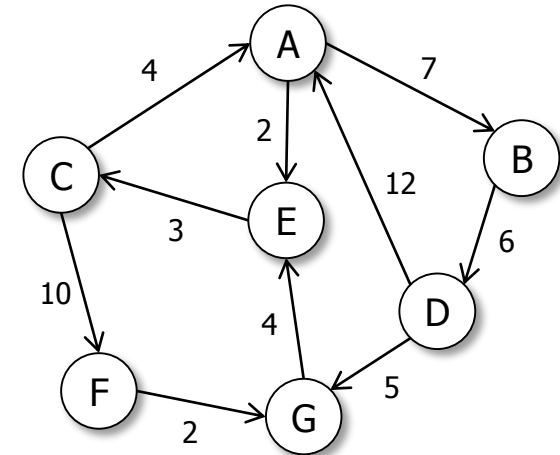


	0	1	2	3	4	5	6	7
0	0	1	0	0	1	0	0	0
1	0	0	0	1	0	0	0	0
2	1	0	0	0	0	1	0	0
3	1	0	0	0	0	0	1	0
4	0	0	1	0	0	0	0	0
5	0	0	0	0	0	0	1	0
6	0	0	0	0	1	0	0	0
7	0	0	0	0	0	0	0	0

➤ Exemplo 3:

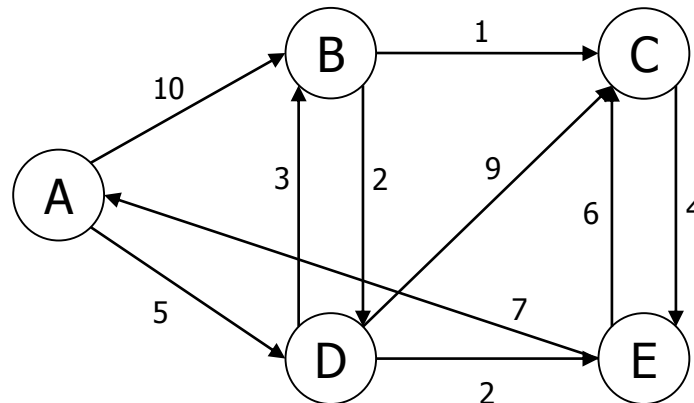
orientado	T
ponderado	T
maxVertices	8
qtdVertices	7
vertices	
matriz	

0	A	F
1	B	F
2	C	F
3	D	F
4	E	F
5	F	F
6	G	F
7		



	0	1	2	3	4	5	6	7
0	0	7	0	0	2	0	0	0
1	0	0	0	6	0	0	0	0
2	4	0	0	0	0	10	0	0
3	12	0	0	0	0	0	5	0
4	0	0	3	0	0	0	0	0
5	0	0	0	0	0	0	2	0
6	0	0	0	0	4	0	0	0
7	0	0	0	0	0	0	0	0

- Caminho de custo mínimo:
 - ✓ Dados dois vértices s e e de um grafo $G(V, A)$ orientado e ponderado, encontre um caminho de s a e em G que tenha o menor custo.
- Exemplo: qual o caminho de custo mínimo de A a C?



- Algoritmo de Dijkstra (Edsger Dijkstra, 1959):
 - ✓ É um dos algoritmos que calcula o caminho de custo mínimo entre vértices de um grafo G ponderado.
 - ✓ Escolhido um vértice v como raiz da busca, este algoritmo calcula o custo mínimo de v para todos os demais vértices de G .
 - ✓ Só funciona se os pesos das arestas não forem negativos.
 - ✓ É bastante simples e com um bom nível de desempenho.



- Algoritmo de Dijkstra (Edsger Dijkstra, 1959): calcular o menor caminho do vértice **s** ao vértice **e** no grafo **G**.

Dijkstra(G, s, e)

Inicializar as estruturas de dados

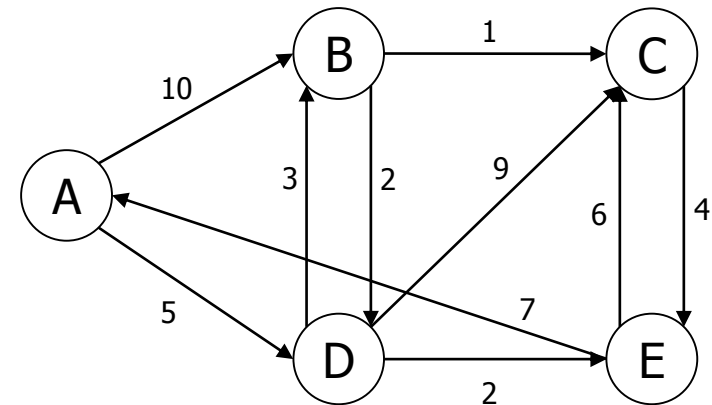
Calcular o custo mínimo de s até os demais vértices de G

Encontrar o caminho de s a e

- Algoritmo de Dijkstra (1ª parte): inicializar as estruturas de dados.
 1. Para todo vértice $v \in G$ faça
 1. $estimativa(v) \leftarrow \infty$
 2. $precedente(v) \leftarrow \text{nulo}$
 2. $estimativa(\text{raiz}) \leftarrow 0$
 3. Crie uma lista L de vértices não visitados com todos os vértices de G

➤ Algoritmo de Dijkstra (1ª parte): melhor caminho de A a C.

1. Para todo vértice $v \in G$ faça
 1. $estimativa(v) \leftarrow \infty$
 2. $precedente(v) \leftarrow \text{nulo}$
2. $estimativa(\text{raiz}) \leftarrow 0$
3. Crie uma lista L de vértices não visitados com todos os vértices de G



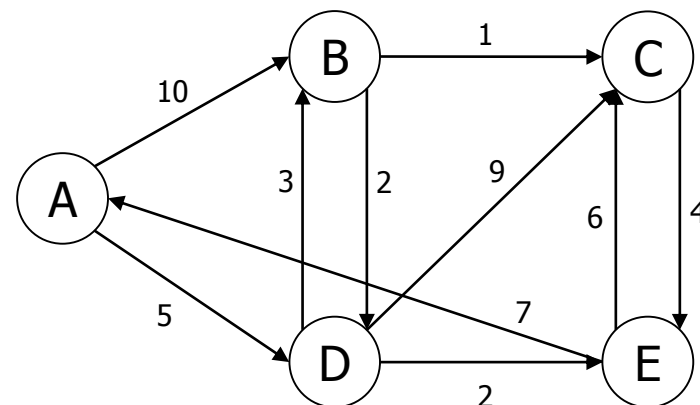
	A	B	C	D	E
Estimativa	0	∞	∞	∞	∞
Precedente	-	-	-	-	-
L	N	N	N	N	N

- Algoritmo de Dijkstra (2ª parte): calcular o custo mínimo de **s** até os demais vértices **G**.

- 4. Enquanto existirem vértices em **L** faça
 - 1. $k \leftarrow$ vértice de **L** com a menor estimativa
 - 2. Exclua **k** de **L**
 - 3. Para todo vértice **j** em **L** que seja sucessor de **k** faça:
 - 1. $S \leftarrow \text{estimativa}(k) + \text{custo}(k, j)$
 - 2. Se $S < \text{estimativa}(j)$ então
 - 1. $\text{estimativa}(j) \leftarrow S$
 - 2. $\text{precedente}(j) \leftarrow k$

➤ Algoritmo de Dijkstra (2ª parte): melhor caminho de A a C.

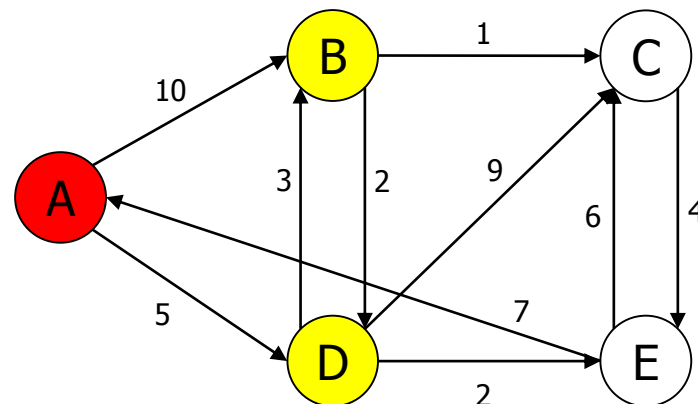
4. Enquanto existirem vértices em L faça
 1. $k \leftarrow$ vértice de L com a menor estimativa
 2. Exclua k de L
 3. Para todo vértice j em L que seja sucessor de k faça:
 1. $S \leftarrow \text{estimativa}(k) + \text{custo}(k, j)$
 2. Se $S < \text{estimativa}(j)$ então
 1. $\text{estimativa}(j) \leftarrow S$
 2. $\text{precedente}(j) \leftarrow k$



	A	B	C	D	E
Estimativa	0	∞	∞	∞	∞
Precedente	-	-	-	-	-
Não visitados	N	N	N	N	N

➤ Algoritmo de Dijkstra (2ª parte): melhor caminho de A a C.

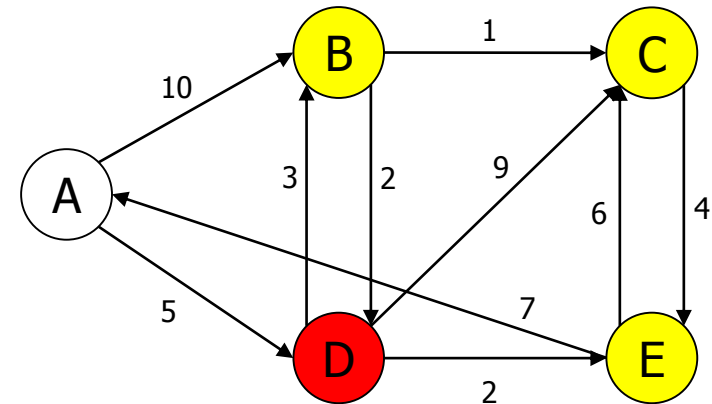
1. A é o vértice de menor estimativa de L ($e = 0$)
2. Excluir A de L
3. Para o vértice B, sucessor de A, faça
 $S \leftarrow 0 + 10$
 Como $S < \infty$ então
 Estimativa(B) $\leftarrow 10$
 Precedente(B) $\leftarrow A$
4. Para o vértice D, sucessor de A, faça
 $S \leftarrow 0 + 5$
 Como $S < \infty$ então
 Estimativa(D) $\leftarrow 5$
 Precedente(D) $\leftarrow A$



	A	B	C	D	E
Estimativa	0	$\infty \rightarrow 10$	∞	$\infty \rightarrow 5$	∞
Precedente	-	A	-	A	-
L	V	N	N	N	N

➤ Algoritmo de Dijkstra (2ª parte): melhor caminho de A a C.

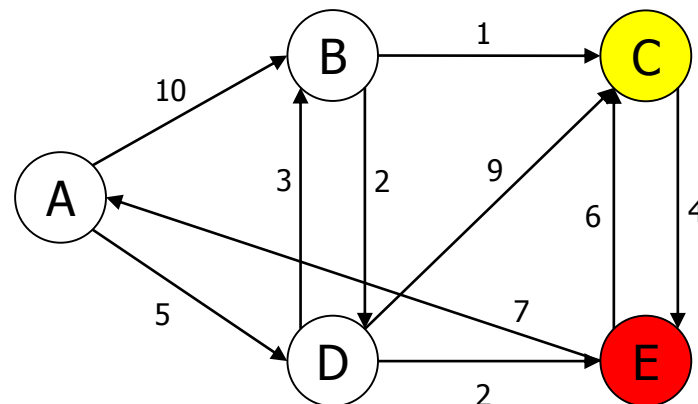
1. D é o vértice de menor estimativa de L ($e = 5$)
2. Excluir D de L
3. Para o vértice B, sucessor de D, faça
 $S \leftarrow 5 + 3$
 Como $S < 10$ então
 Estimativa(B) $\leftarrow 8$
 Precedente(B) $\leftarrow D$
4. Para o vértice C, sucessor de D, faça
 $S \leftarrow 5 + 9$
 Como $S < \infty$ então
 Estimativa(C) $\leftarrow 14$
 Precedente(C) $\leftarrow D$
5. Para o vértice E, sucessor de D, faça
 $S \leftarrow 5 + 2$
 Como $S < \infty$ então
 Estimativa(E) $\leftarrow 7$
 Precedente(E) $\leftarrow D$



	A	B	C	D	E
Estimativa	0	10 → 8	$\infty \rightarrow 14$	5	$\infty \rightarrow 7$
Precedente	-	D	D	A	D
L	V	N	N	V	N

➤ Algoritmo de Dijkstra (2ª parte): melhor caminho de A a C.

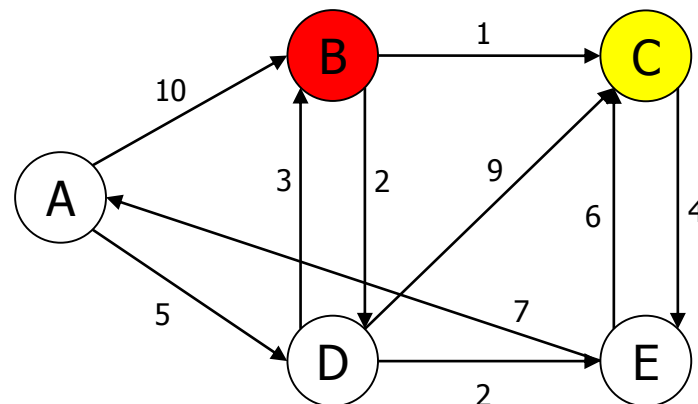
1. E é o vértice de menor estimativa de L (e = 7)
2. Excluir E de L
3. Para o vértice C, sucessor de E, faça
 $S \leftarrow 7 + 6$
 Como $S < 14$ então
 $Estimativa(C) \leftarrow 13$
 $Precedente(C) \leftarrow E$



	A	B	C	D	E
Estimativa	0	8	14→13	5	7
Precedente	-	D	E	A	D
L	V	N	N	V	V

➤ Algoritmo de Dijkstra (2ª parte): melhor caminho de A a C.

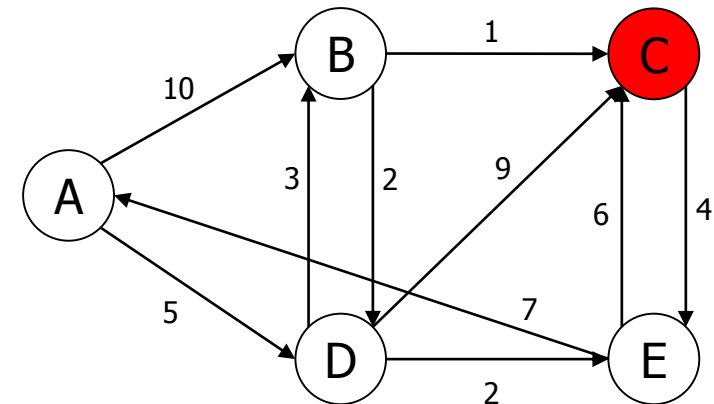
1. B é o vértice de menor estimativa de L (e = 8)
2. Excluir B de L
3. Para o vértice C, sucessor de B, faça
 $S \leftarrow 8 + 1$
 Como $S < 13$ então
 $\text{Estimativa}(C) \leftarrow 9$
 $\text{Precedente}(C) \leftarrow B$



	A	B	C	D	E
Estimativa	0	8	13→9	5	7
Precedente	-	D	B	A	D
L	V	V	N	V	V

➤ Algoritmo de Dijkstra (2ª parte): melhor caminho de A a C.

1. C é o vértice de menor estimativa de L (e = 9)
2. Excluir C de L
3. Como C não tem sucessores em L então o loop termina

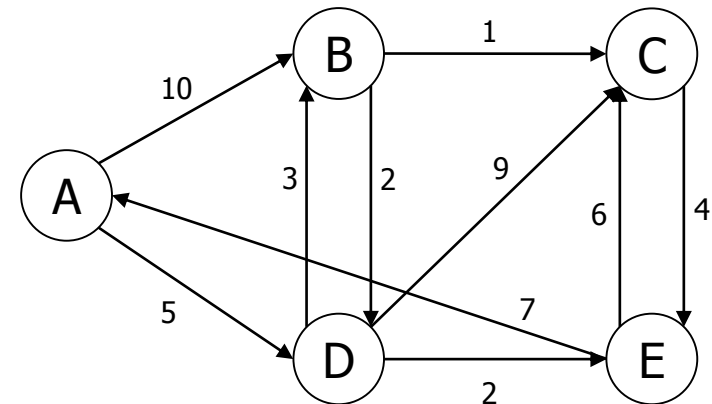


	A	B	C	D	E
Estimativa	0	8	9	5	7
Precedente	-	D	B	A	D
L	V	V	V	V	V

➤ Algoritmo de Dijkstra (2ª parte): melhor caminho de A a C.

➤ Ao final da 2ª parte do algoritmo, os valores das estimativas na tabela representam o **custo** do vértice **raiz** até determinado vértice:

- ✓ Caminho de A a B custa 8
- ✓ Caminho de A a C custa 9
- ✓ Caminho de A a D custa 5
- ✓ Caminho de A a E custa 7



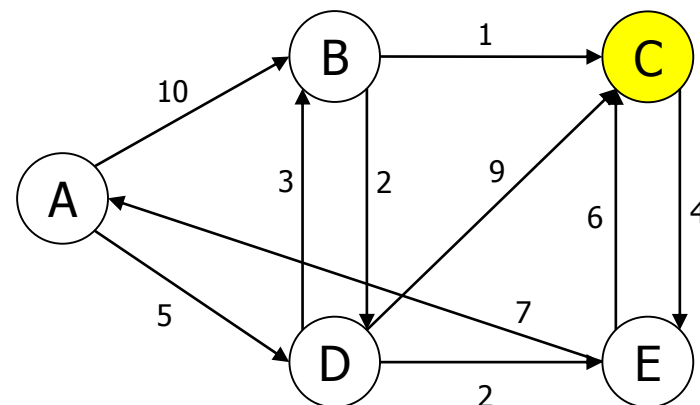
	A	B	C	D	E
Estimativa	0	8	9	5	7
Precedente	-	D	B	A	D
L	V	V	V	V	V

➤ Algoritmo de Dijkstra (3ª parte): encontrar o caminho de s a e .

5. Caminho $\leftarrow \{e\}$
6. $p \leftarrow e$
7. Enquanto $p \neq s$ faça
 1. $p \leftarrow \text{precedente}(p)$
 2. Caminho $\leftarrow p + \text{Caminho}$

- Algoritmo de Dijkstra (3ª parte): melhor caminho de A a C.

Caminho $\leftarrow \{ C \}$



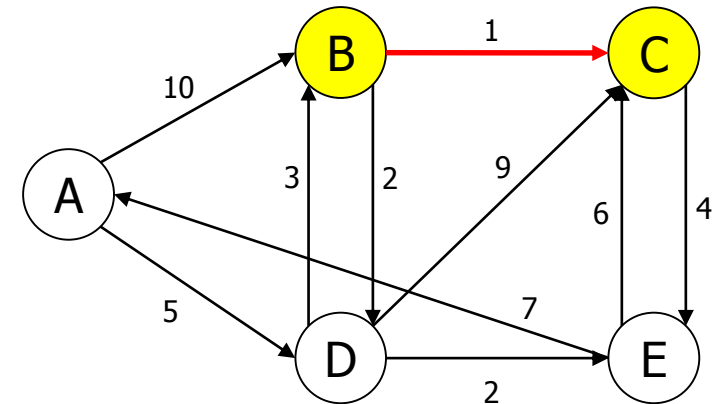
	A	B	C	D	E
Estimativa	0	8	9	5	7
Precedente	-	D	B	A	D
L	V	V	V	V	V

- Algoritmo de Dijkstra (3ª parte): melhor caminho de A a C.

Caminho $\leftarrow \{ C \}$

B é precedente de C

Caminho $\leftarrow \{ B, C \}$



	A	B	C	D	E
Estimativa	0	8	9	5	7
Precedente	-	D	B	A	D
L	V	V	V	V	V

- Algoritmo de Dijkstra (3ª parte): melhor caminho de A a C.

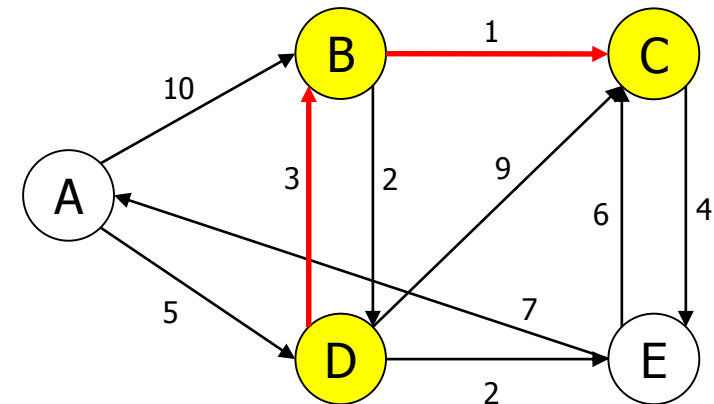
Caminho $\leftarrow \{ C \}$

B é precedente de C

Caminho $\leftarrow \{ B, C \}$

D é precedente de B

Caminho $\leftarrow \{ D, B, C \}$



	A	B	C	D	E
Estimativa	0	8	9	5	7
Precedente	-	D	B	A	D
L	V	V	V	V	V

- Algoritmo de Dijkstra (3ª parte): melhor caminho de A a C.

Caminho $\leftarrow \{ C \}$

B é precedente de C

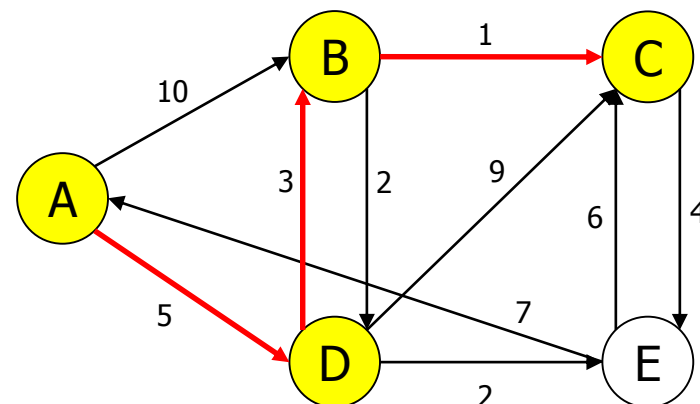
Caminho $\leftarrow \{ B, C \}$

D é precedente de B

Caminho $\leftarrow \{ D, B, C \}$

A é precedente de D

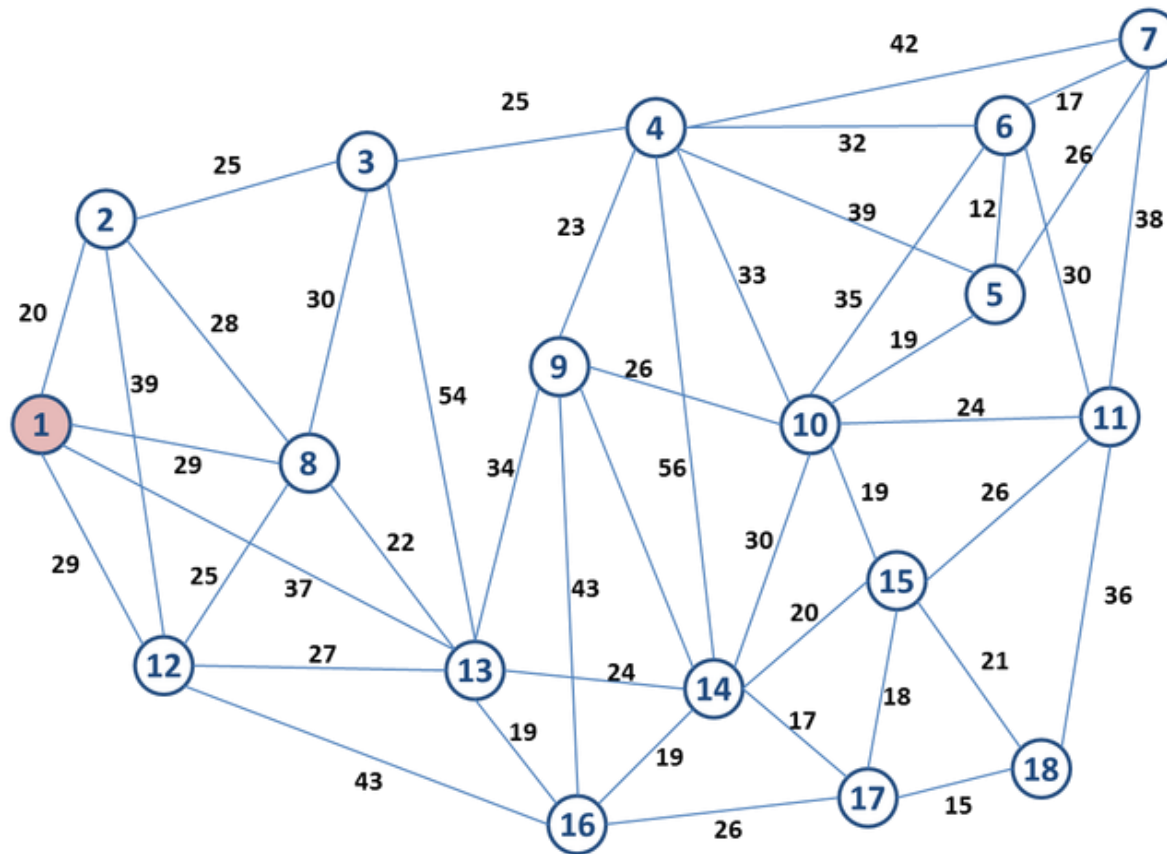
Caminho $\leftarrow \{ A, D, B, C \}$



	A	B	C	D	E
Estimativa	0	8	9	5	7
Precedente	-	D	B	A	D
L	V	V	V	V	V

- Um caixeiro viajante necessita visitar um certo número de cidades e retornar à cidade de origem de tal maneira que cada cidade é visitada exatamente uma vez e que a distância total percorrida seja a menor possível. Dada a distância entre as cidades, que rota ele deve escolher?
- É possível representar esse problema usando um **grafo orientado e ponderado**:
 - ✓ Vértices: cidades.
 - ✓ Arestas: estrada que liga as cidades.
 - ✓ Peso: distância entre as cidades.

- O PCV pode ser traduzido como o problema de encontrar um **ciclo Hamiltoniano** de custo mínimo.



- A princípio, o PCV pode ser resolvido determinando-se todas as rotas possíveis e escolhendo aquela com a menor distância.
- Considere que o problema envolva 10 cidades. O número total de possíveis rotas seria de $9! = 362.880$.
- Um computador capaz de examinar 100 milhões de rotas/segundo levaria 0,0036 s para encontrar a melhor rota.
- Entretanto, se dobrarmos o número de cidades para 20, teremos $19!$ ou $1,22 \times 10^{17}$ rotas. Usando o mesmo computador levaríamos **38 anos** para encontrar a melhor rota!
- Soluções adotadas:
 - ✓ Algoritmo baseado em árvore geradora mínima
 - ✓ Algoritmos melhorativos como 2-opt ou 3-opt
 - ✓ Algoritmos do tipo "branch and bound"
 - ✓ Algoritmos genéticos
 - ✓ dentre outros.